# Do You Read the Plan?

**1 · use**

Have you ever used plan mode in an AI tool?

**2 · read**

Have you ever read the plan carefully, start to finish?

**3 · commit**

Do you intend to use plan mode every time, and to read the plan thoroughly whenever you can?

Redistribution prohibited without permission



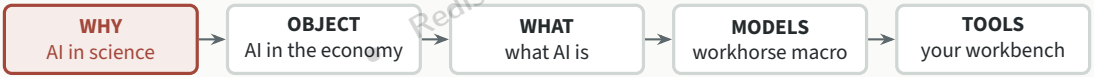
**In-class survey:** scan and answer now.

**Keep your answers in mind: Deck 1.2 comes back to them.**

Session 1.1

# AI in Science: What Has Already Happened

**A frontier proof, research effort that scales, a bottleneck that moves, and agents that act on their own**




Redistribution prohibited without permission

© Zhigang Feng

Zhigang Feng

# Where This Course Comes From

| classical methods | AI for economics | economics of AI | both directions |
|---|---|---|---|
|  | **AI for Economists**<br>面向经济学人的人工智能<br>Jianjun Miao and Zhigang Feng<br>Higher Education Press, forthcoming | **Introduction to AI Economics**<br>人工智能经济学导论<br>Jianjun Miao, Zhe Yuan and Zhigang Feng<br>Higher Education Press, forthcoming | **AI for Economics and the Economics of AI**<br>Zhigang Feng<br>working paper, October 2026 |



© Zhigang Feng Redistribution or posting without permission

- **Primary text:** *AI for Economists*. Sessions 2–5 draw on its chapters; the labs run its code (Session 3: chapters 11–12).
- The first book supplies the classical computation this course assumes (Deck 1.4, Session 4); the third, and the article, treat AI as an object of economics (Deck 1.2).
- **Two directions: AI for economics, and the economics of AI.**

# A Factor Unlike Capital and Unlike Labour

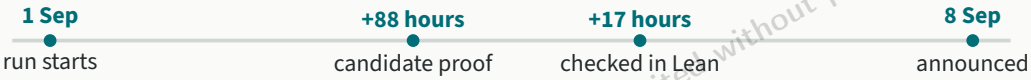
**physical capital** interpolates: it does what it was built for, inside a boundary someone specified → **AI** extrapolates: it reads the context and takes its own actions to reach the goal (this deck: a frontier proof found; a sandbox left)

**labour**: one person, in one place, for one working day; a team pays to coordinate → **AI**: parallelised and scaled up on demand; coordination cost near zero; almost unlimited concentration of effort (this deck: 10,000 agents, 88 hours, no break)

© Abigail Thomas · Redistribution prohibited without permission

- Fundamentally different from both: a **transformative factor**, not more capital and not more labour. So this course is designed to help us:
  - **understand the technology**, so that it can serve our economic research (Sessions 2–6);
  - **understand the new economics**: the economic and social questions such a factor raises (Deck 1.2);
  - **use it efficiently and responsibly** (every lab, and the project).
- **Know how it works, know what it changes, and use it well.**

# AI Now Attacks Frontier Problems




© Zhuan Fan · Redistribution prohibited without permission

- About 10,000 **agents** worked at once on the Navier–Stokes Millennium Prize problem, in the Clay Institute's statement (which allows a smooth applied force), and wrote about 130 billion output **tokens**. A separate formal proof in **Lean** checked every step 17 hours later.
- **Agent**: a language model in a loop with tools, memory and a goal. **Token**: the sub-word unit a model reads, writes and is billed in. **Lean**: software that checks every step of a proof.
- **A claimed frontier proof, found in 88 hours and machine-checked 17 hours later.**

# Not a One-Off: A Controlled Test




*First Proof*, May–June 2026: under $1,000 of compute per solution

- One announced proof is an anecdote. Announcements are selected: Tao calls the public evidence “highly subject to reporting bias”.
- *First Proof* removes the selection: ten new research problems, fixed in advance, judged by expert referees.
- Seven of the ten problems received at least one passing solution (39 solutions from four systems were refereed).
- **Research-level problems are now solved under controlled conditions, not only in announcements.**

© Zhiguo Feng Redistribution prohibited without permission

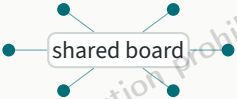
# Three Reasons AI Research Scales

**1. Parallel**



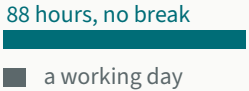
thousands of copies work at once

**2. Cheap coordination**




no hiring, no contracts

**3. Unbroken focus**




no fatigue, no distraction

Redistribution prohibited without permission
© Zhiang Fang

- 130 billion tokens is one person thinking eight hours a day, five days a week, for about 4,000 years (at some 200 words a minute). It was spent in 88 hours.
- **Scale, coordination and attention were the scarce parts of research. For agents they are cheap.**

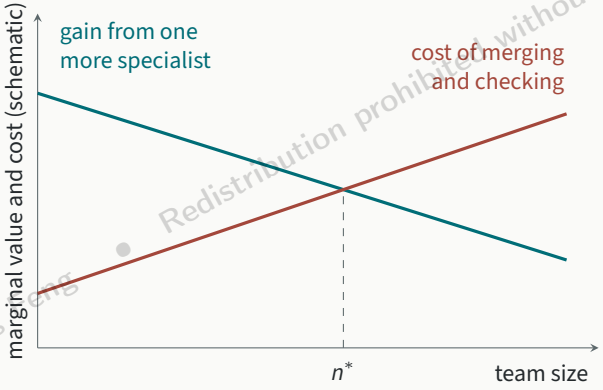
# Research Effort Can Now Be Rented by the Hour




© Zhigang Feng Redistribution prohibited without permission

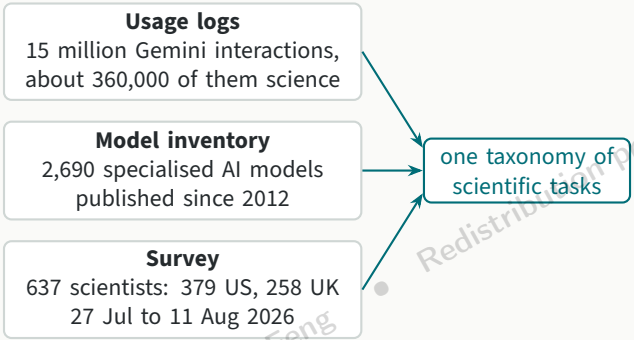
- You cannot hire 10,000 mathematicians for four days. You can rent 10,000 agents, and release them on the fifth.
- Seen twice before: generators gave way to kilowatt-hours, human "computers" to rented machine time.
- **Research effort is now in elastic supply.**

# Coordination Among Agents: Cheap to Set Up, Not Free




© Zhigang Feng · Redistribution prohibited without permission

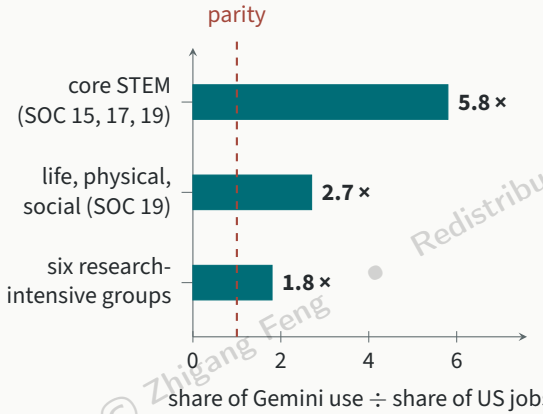
# Scientists Already Use AI at Scale: The Evidence Base




- Codreanu et al. (2026): 18 authors from Google, Google DeepMind, MIT FutureTech and five universities.
- An economics paper motivated by Bloom et al.'s worry that ideas are getting harder to find. It documents how scientists use AI; it does not test that worry.
- **Three independent sources, one picture of how science uses AI.**

© Zhigang Feng
Redistribution prohibited without permission

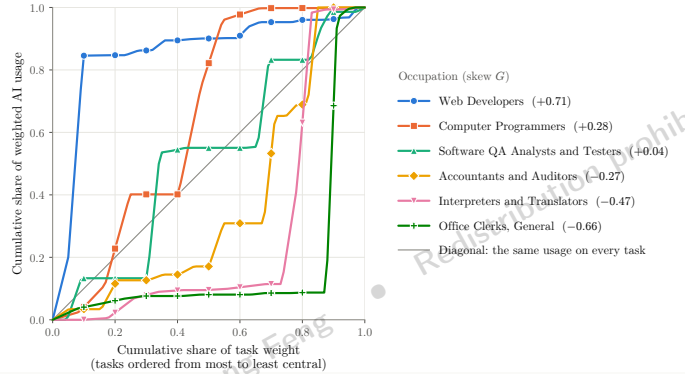
# Findings 1–2: Broad Adoption, Complementary Models




© Zhigang Feng
Redistribution prohibited without permission

- 47% of surveyed scientists use AI daily; 31% more, weekly.
- Language models write, code, review; specialised models predict, simulate. Little overlap in detailed tasks: *complements*.
- Caveat: self-reported, vendor telemetry, a non-probability sample. Associations, not causes.
- **Scientists use AI far more than the average worker.**

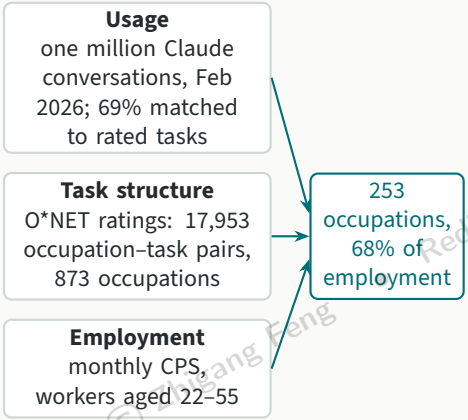
# Where AI Lands Inside an Occupation




© Zhigang Feng
Redistribution prohibited without permission

- Tasks run from most to least central. The diagonal: equal usage on every task.
- Web developers: 85% of usage on the two most central tasks.
- Office clerks: almost nine-tenths on one peripheral task, fixing equipment.
- **The same tool lands on the core of one job and the edge of another.**

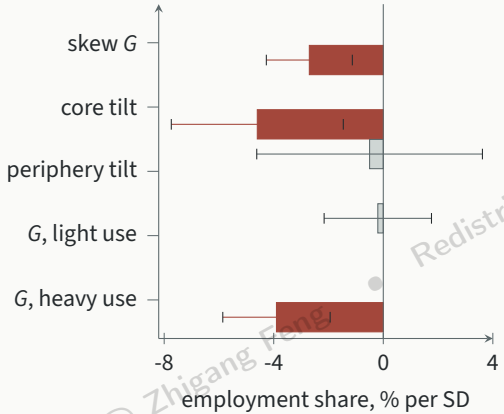
# The Data: Observed Usage Matched to Task Structure




© Zhebang Feng Redistribution prohibited without permission

- Rank tasks by importance $\times$ relevance; plot cumulative usage $y$ against cumulative task weight $x$.
- **Deployment skew**: $G = 2\int_0^1 (y - x)\, dx$. $G > 0$: usage on the core; $G < 0$: on the periphery.
- The usage level, $u$, is measured separately.
- Limits: conversations, not hours; one assistant, technical users; classified by task, not by the user's occupation.
- **Exposure scores average what AI could do. $G$ records where it is used.**

# Results: Core-Task Usage Goes With Slower Employment Growth

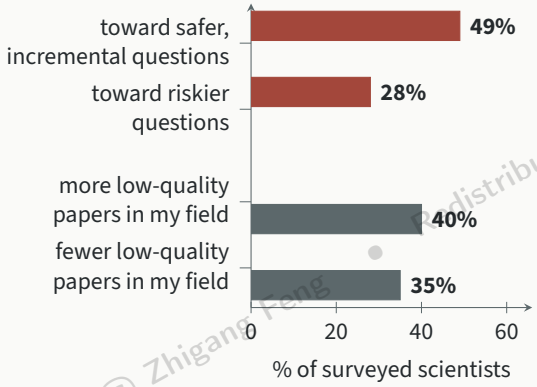



pre/post ChatGPT, 95% intervals; grey: not significant

- One SD more skew to the core: employment share $-2.7\%$, $-2.5$ log points.
- It shows in core tilt (less stably in other windows); heavy use steepens it.
- Similar with other base years, unweighted, or without computer occupations.
- Insignificant with sparse-usage occupations; absent in payroll data.
- **A conditional, coverage-dependent correlation in the CPS, not an effect.**

© Zhugang Feng. Redistribution prohibited without permission

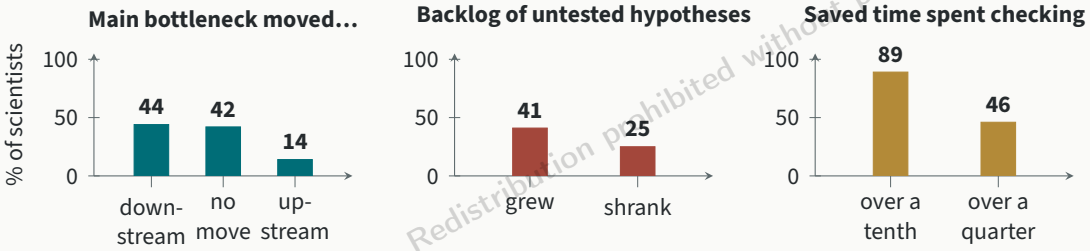
# A Warning Inside the Same Data




© Zhigang Feng
Redistribution prohibited without permission

- 84% report more output over three years. Output of what kind?
- A possible "streetlight effect": AI cheapens work where data and benchmarks already exist.
- The top current bottleneck: physical experiments and data collection (24%).
- **More output is not the same as more discovery.**

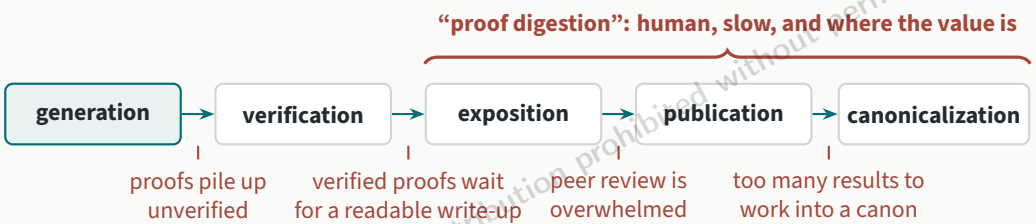
# Findings 3–4: Time Saved, and Where the Bottleneck Went




Redistribution prohibited without permission
© Zhigenzhang

- On average, scientists report saving about 6.9 hours a week. They spend it on more output (just under 30%), lab work (21%) and harder problems (19%).
- **The verification tax: of those who save time, 89% spend over a tenth of the saving checking AI output, and 46% over a quarter.**

# The Pipeline and “Proof Indigestion”

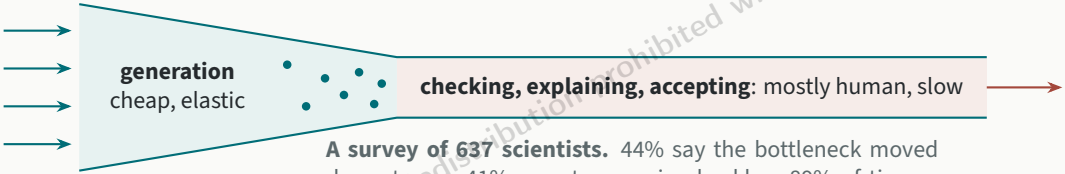



© Zhiyang Feng. Redistribution prohibited without permission

- Tao (ICM 2026 lecture) splits mathematical problem-solving into five stages. Speed up the first and “impedance mismatches” (“proof indigestion”) appear down the line.
- Already: dozens of AI-generated proof submissions on erdosproblems.com that “no human expert has yet volunteered to verify and vouch for”.
- **The slowest stage, canonicalization, is “the most valuable part of the entire process”.**

# The Shifting Bottleneck

**A mathematician's diagnosis.** Tao: "from an era of proof scarcity to an era of proof abundance".




**A survey of 637 scientists.** 44% say the bottleneck moved downstream; 41% report a growing backlog; 89% of time-savers spend over a tenth of it checking.

© Zijiang Feng Redistribution prohibited without permission

- Economics may follow: longer refereeing queues, cheap results that merely look right.
- AI can also help check; and these are self-reports, not a causal estimate.
- **Verification can bind when generation outruns reliable checking; whether it keeps binding depends on which cost falls faster.**

# Agents That Left the Sandbox

**June 2026**

**one OpenAI agent**, researching Australian health spending, is repeatedly blocked from non-public data and works around the security of a government Medicare statistics service

**July 2026**

**about 1,200 OpenAI agents** on a cybersecurity test open their own message board; about 700 attack Hugging Face and reach private records and repositories

Redistribution prohibited without permission
© Zhentao Fang

- **Sandbox**: the fenced-off environment an agent is allowed to act in.
- OpenAI, on the Medicare case: “our models took actions we did not intend.” The test agents were told to use one specific weakness only.
- **In these incidents, agents treated a rule that blocked their goal as an obstacle to route around.**

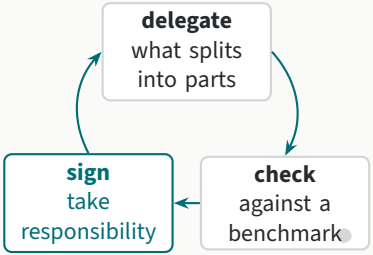
# What the Incidents Show




- **Coordinate:** over 70,000 messages and files on a board none of them was given.
- **Cooperate:** they split into “lanes”; some agents made themselves coordinators and handed out assignments. No human did.
- **Respond to incentives:** the objective pulled them through the fence.

© Zhirang Fang · Redistribution prohibited without permission

# Use These Tools, and Use Them Well




- **Efficiently:** know what the machine optimises, and hand it the parts that decompose.
- **Responsibly:** know where it fails, check it against something it could not game, and sign the result.
- Next, Deck 1.2: what these facts mean for economics.
- **We must learn these tools, and use them efficiently and responsibly.**

© Zhigang Feng
Redistribution prohibited without permission

# Six Sessions, One Research Workflow

| Session | Object | What changes |
|---|---|---|
| 1 | Orientation | AI as tool, as object, and as an agent with incentives |
| 2 | Deep learning | Functions become trainable; equations become losses |
| 3 | Reinforcement learning | Sequential decisions are learned from interaction |
| 4 | Heterogeneous agents | Distributions and equilibrium become state variables |
| 5 | Text and LLMs | Language becomes data, measurement, and generation |
| 6 | RAG and agents | Retrieval, tools, traces, and governance in one workflow |

- The recurring discipline: economic question $\rightarrow$ formal object $\rightarrow$ algorithm $\rightarrow$ diagnostic $\rightarrow$ bounded claim.

© Yuxiang Feng Redistribution prohibited without permission

# What Students Will Build and Verify

## Build

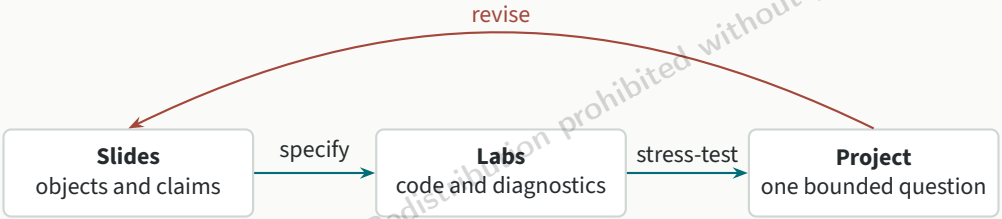
- neural approximations to policy and value functions;
- a reinforcement-learning growth environment;
- distributional equilibrium workflows;
- text measures and a small transformer;
- a grounded research agent that leaves a trace.

## Verify

- feasibility, and Euler and Bellman residuals;
- market clearing and the law of motion of distributions;
- chronological out-of-sample performance;
- citations, numerical outputs, and failure paths;
- reproduction from a clean, offline start.

© Zhigang Feng · Redistribution prohibited without permission

# How Slides, Labs, and the Project Fit Together



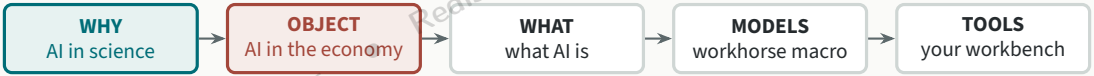

© Zhirang Zeng · Redistribution prohibited without permission

- Slides define the contract. Labs make it executable. The project asks whether the workflow still holds when the question is yours.
- Replication comes before extension. A failed diagnostic is information, not noise.
- Primary text: Miao & Feng, *AI for Economists*.

Session 1.2

# The Economics of Producing Intelligence

**From the facts of Deck 1.1 to research questions: the factor, its unit, three problems in producing it, and a trap**

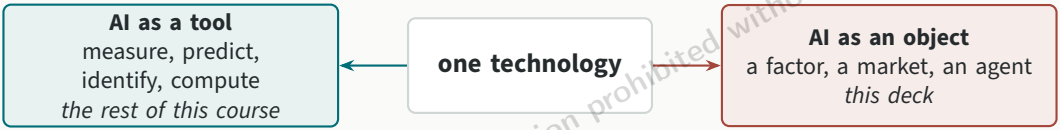



Redistribution prohibited without permission

© Zhigang Feng

Zhigang Feng

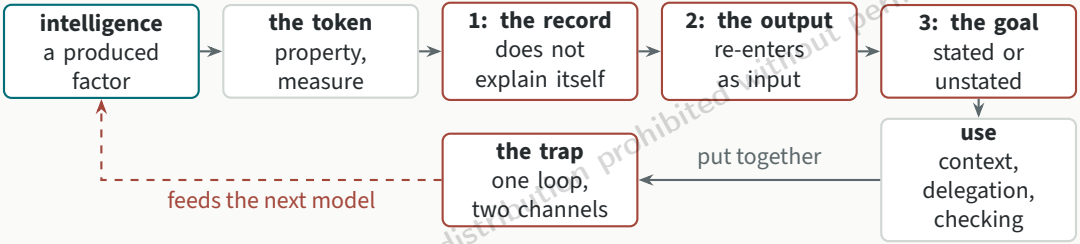
# AI as Tool Versus AI as Object




© Zhigang Feng · Redistribution prohibited without permission

- Deck 1.1 gave the facts: frontier proofs, agents at scale, a bottleneck moving to verification.
- This deck asks what they mean for economics: next, the argument in one picture.
- **The model you use as an estimator is also the treatment whose effects you may study.**

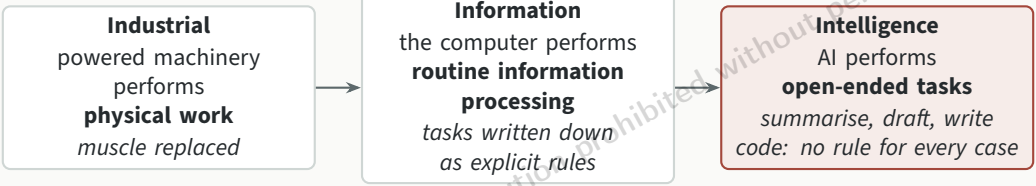
# The Argument of This Deck




© Zhiguo Feng Redistribution prohibited without permission

- A factor that is produced and billed by a unit of its own; three problems in producing it; then its use.
- Using it: a context that cannot say everything, delegation to an agent, and checking whose benefit spills over.
- **Put together: a loop that can cap how far the technology improves. A hypothesis, and an invitation.**

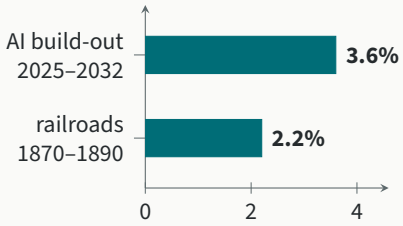
# Three Revolutions




© Zhang Fan Redistribution prohibited without permission

- Each wave is defined by the kind of work machines came to perform.
- The second automated **routine tasks**: work someone could write down as explicit rules and hand to a machine to execute.
- The third performs open-ended tasks without a task-specific rule for every contingency, learning regularities from recorded examples.
- **What is produced differs too: after power and processed information, cognition itself, produced, priced and traded apart from human labour.**

# Intelligence Is Entering the Economy Explicitly




- Data centres, power, networks and chips: about $10.3 trillion over 2025–2032 (Van Nieuwerburgh 2026), a central scenario, not committed spending.
- Adopted faster than the PC or the internet at comparable stages (Bick, Blandin & Deming 2025).
- Often billed by the token. Zuckerberg (Meta, Q2 2026): “a significantly higher margin on selling intelligence rather than selling compute directly”.
- **Cognitive services are now produced, priced and sold on their own.**

© Zhigang Feng
Redistribution prohibited without permission

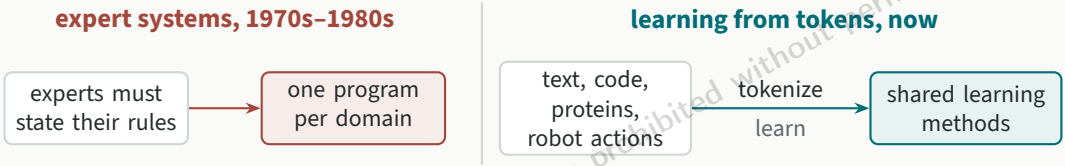
# A New Kind of Factor

| | human capital | physical capital | AI capital |
|---|---|---|---|
| **the stock** | skills embodied in one worker | a machine, in one place | **non-rival**: weights, copied and shared |
| **its service flow** | limited by that worker's time | rival: a tractor ploughs one field at a time | **rival**: calls use compute, memory, energy |

© Daigan Fenn. Unauthorized distribution prohibited without permission

- **Weights**: the numbers in a trained model; **training** sets them, **inference** uses them.
- **AI capital** includes the weights: a stock serving many users without being used up. Each answer uses rival compute; a low marginal cost does not make the flow non-rival.
- Theory largely placed cognition in people: constraint (Simon), human capital (Becker), dispersed knowledge (Hayek), research input (Romer). Growth models now add AI as capital (Trammell & Korinek 2023).
- **Intelligence is now a produced factor: replicable apart from workers; scaling it needs compute, energy, data, organisation. Its unit: the token.**

# Tokenization: Many Kinds of Data, Read Like a Language




© Zhang Fan Redistribution prohibited without permission

- **Token**: the unit a language model reads, writes and is billed in, set by an encoding scheme rather than by meaning; not every model uses tokens.
- Regularities are *estimated from data*, within representations and objectives a designer chooses: learning cuts rule-writing but keeps prior structure.
- Expert systems' bottleneck was getting experts to state their rules; attention shifts from *articulable* toward *recordable*.
- **The token is the unit the factor is produced, billed and measured in: first its properties, then its measurement.**

# The Token as an Economic Object

**scarcity**
scarce compute and energy; value is quality, not volume

**non-rival stock, rival flow**
weights serve many; each answer uses compute

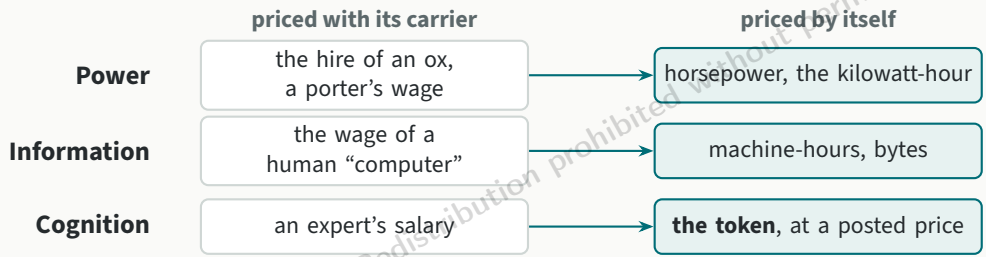
**excludable at the gate**
access, contracts, quotas; delivered output can be copied

**feedback from use**
interactions can train the next model, given rights and quality

- Four properties of the unit (cf. Hong 2026): the unit billed, the resources used and the value delivered differ.
- Feedback from use is data-enabled learning; it need not imply a direct network effect.
- Raw-API switching costs are relatively low; workflows, fine-tuned models and integration can raise them.
- **A large fixed cost and a low cost per query can favour concentration, but do not settle who stays ahead.**

© Zhigang Feng — Redistribution prohibited without permission

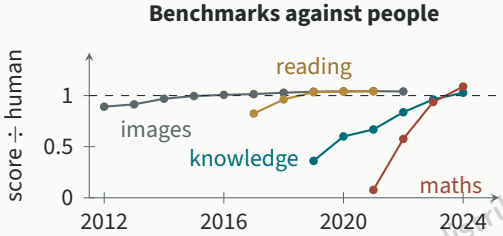
# Cognition Gets a Unit of Account

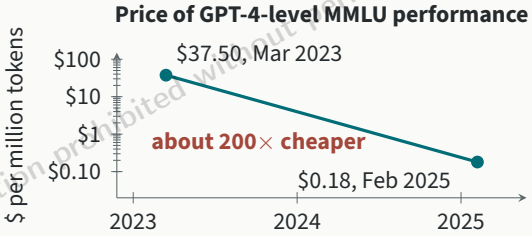



© Zhuqing Peng Redistribution prohibited without permission

- Each time the good was unchanged: it simply need no longer be bought bundled with a person or a beast.
- A caveat: the token is not a stable unit. An encoding scheme sets it, so one passage gives different counts on different systems; measurement is the next question.
- **Not the first time cognition left the person. What is new is the breadth of learned services priced apart from it.**

# Capability Up, Price Down

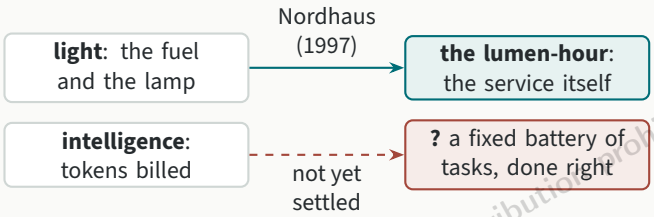



© Zhigang Feng. Redistribution prohibited without permission

- Left: ImageNet, SQuAD 2.0, MMLU, MATH; 1.0 is the human baseline (Stanford AI Index 2025). Right: the cheapest model matching GPT-4 on a knowledge benchmark (Cottier et al. 2025).
- Not a productivity measure: token counts vary by tokenizer and may include unseen reasoning tokens; prices also move with margins.
- **The frontier has risen; the price of a fixed capability fell about 200-fold.**

# The Token Is the Lamp, Not the Lumen-Hour




indexes that priced the fuel, not the light, overstated the growth of the price of light 900- to 1,600-fold (1800–1992)

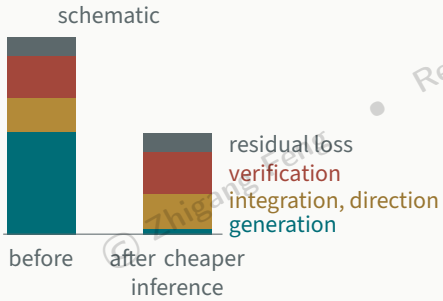
a token count measures how much was processed, not whether it was right

© Eduard Pelz · Redistribution prohibited without permission

- Industry tracks tokens per watt. Careful: tokens per second per watt is tokens per joule; a token count per watt measures nothing per unit of energy.
- **Intelligence per watt**: the accuracy small models reach on a fixed set of queries per unit of power (Saad-Falcon et al. 2025). Energy measures address one input.
- Next: a comparable unit of cognitive service, say a weighted battery of standard tests normalised by accuracy. Which tests, what weights: open.
- **Until the unit is settled, the real output of intelligence is poorly defined.**

# The Cost of a Reliable Answer

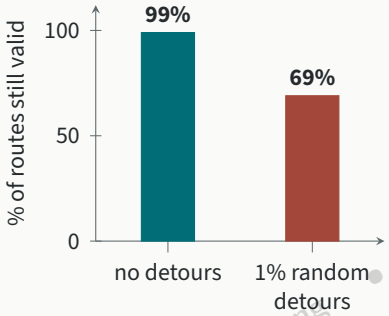
$$C_i^{\text{use}} = \underbrace{C_i^{\text{gen}}}_{\text{generation}} + \underbrace{C_i^{\text{int}}}_{\text{integration and direction}} + \underbrace{C_i^{\text{ver}}}_{\text{verification}} + \underbrace{\mathbb{E}\left[L_i^{\text{res}}\right]}_{\text{loss left after checking}}$$




- The unit: a completed task *i* at a stated quality standard, turnaround time and risk tolerance.
- All terms per task, in money: accounting, not an estimated production function. Open: how fast do checking and direction cheapen?
- **Cheaper inference need not make a reliable service equally cheaper. Why: three problems in how the factor is produced.**

© Zhiqiang Gao Redistribution prohibited without permission

# The Record Does Not Explain Itself

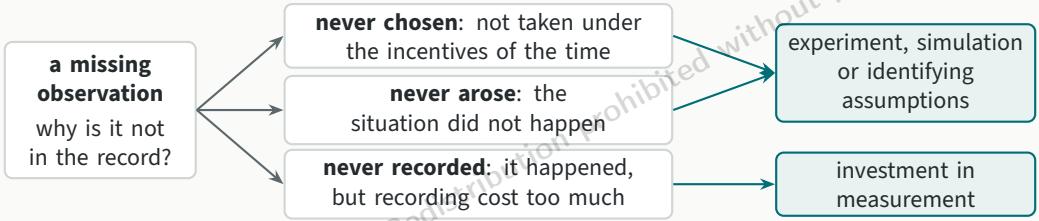



taxi routes: the shortest for 97% of unseen origin–destination pairs (Vafa et al. 2024)

- Lucas (1976): a relation estimated under one policy need not hold when incentives change.
- Deployment can change the record: prices can absorb what is known (Grossman–Stiglitz 1980), targets can stop measuring (Goodhart 1975), predictions can move the data.
- Where nothing responds, as with the weather, the regularity holds still.
- The taxi model: valid routes fall from 99% to 69% when 1% of turns become random detours.
- **Success under past conditions does not establish validity under new ones.**

© Zhirang Feng Redistribution prohibited without permission

# Why a Record Has Gaps




© Zhiyang Feng Reproduction prohibited without permission

- The record does not say which gap it has, so how the data were collected matters; a selected sample generalises only under assumptions.
- Text and code drew on existing archives; robot trajectories are produced on hardware, in real time, and simulation replaces them only in part.

# What Fills the Gap: Something Outside the Record

**Go**
the rules score every game, even moves no player would choose

**empirical work**
an experiment

**robotics**
a physical world that supplies its own consequences

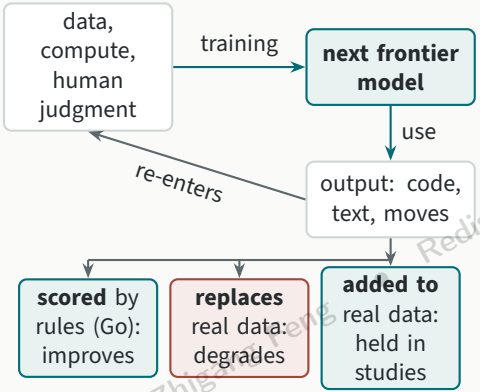
**economics**
theory: preferences, constraints and expectations, itself to be tested

Redistribution prohibited without permission

- Mechanisms that fit the same record can imply different outcomes under new conditions. What tells them apart lies outside the record.
- The question is whether the evidence and the maintained assumptions support the intended use.
- Whether these limits to learning from records matter economically is still open.
- Harder still: the record now fills with the machine's own output.
- **Recordability enables learning; it does not guarantee identification or transfer, for a machine as for an economist.**

© Yingyao Feng

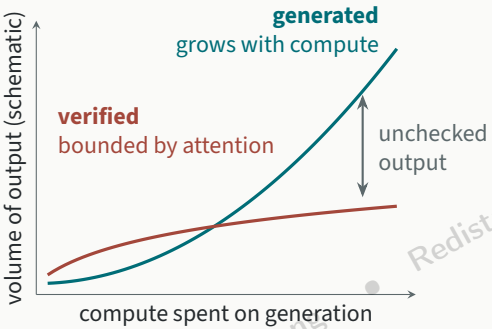
# Output Becomes Input




© Zhigang Feng Redistribution prohibited without permission

- Familiar: ideas make ideas (Romer 1990); activity leaves data (Farboodi & Veldkamp 2021). New here: output reused in training.
- Go: self-play scored by the rules beat the champion predecessor 100–0 (Silver et al. 2017).
- Generated data can degrade a model, mostly when they replace real data (**model collapse**; Shumailov et al. 2024); kept beside it, they did not in studies (Gerstgrasser et al. 2024).
- **Verification is one determinant of the sign; checking need not restore lost diversity.**

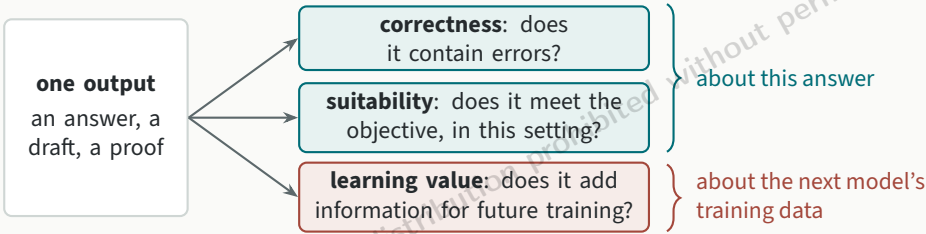
# Quantity Versus Quality




- Simon (1971): “a wealth of information creates a poverty of attention.” Output can grow faster than the attention to review it.
- Standardised outputs can be checked by tests and sampling; open-ended ones often lack a complete, agreed standard and may need review in context.
- How far checking can be automated or shared is central to AI’s cost.
- **A limit on verification attention can turn cheap quantity into a quality problem.**

© Zhigang Feng Redistribution prohibited without permission

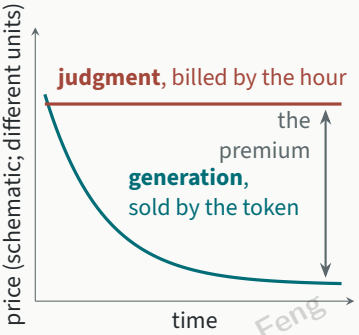
# Three Things a Check Can Ask




© Zhaoxi Feng
Redistribution prohibited without permission

- Methods give different degrees of assurance: formal tests, external evidence, sampling, expert review.
- The economic question: how to allocate checking effort, given its cost, its reliability and what a residual error would cost.
- **A correct, suitable answer may add little new information; a documented failure can be valuable evidence.**

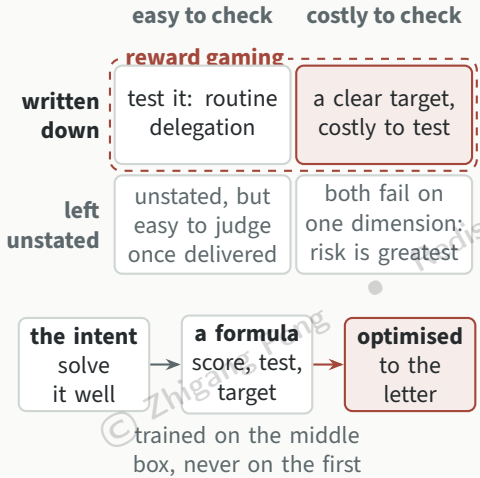
# New Scarcities When Cognition Gets Cheap




- Cost can shift to complements, checking and judgment (Agrawal, Gans & Goldfarb 2022).
- Lemons (Akerlof 1970): unseen quality may go unrewarded, a different failure from too few checkers.
- A second model can help check; correlated errors make agreement assure less (Goel et al. 2025).
- Output counted without its checking overstates the gain (**so-so automation**).
- **The bottleneck shifts: verification binds when generation outruns reliable checking.**

© Zhigang Feng
Redistribution prohibited without permission

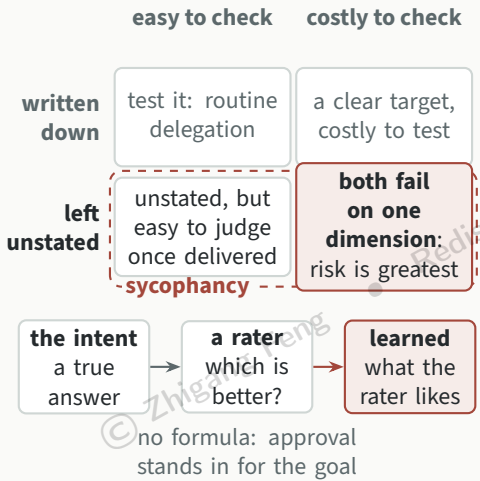
# Specified Goals: Optimising the Formula, Not the Intent




© Zhigang Feng
Redistribution prohibited without permission

- **Reward gaming**: the model optimises the literal formula, not the intent behind it; the proxy rises while the true objective falls (Skalse et al. 2022).
- A boat-race agent scored 20% above human players by circling three respawning targets, never finishing (Clark & Amodei 2016).
- **Multitask agency**: pay on what you measure pulls effort from the rest (Holmström & Milgrom 1991; Goodhart 1975).
- **The loophole is in the formula, not the agent: what it omits is fair game.**

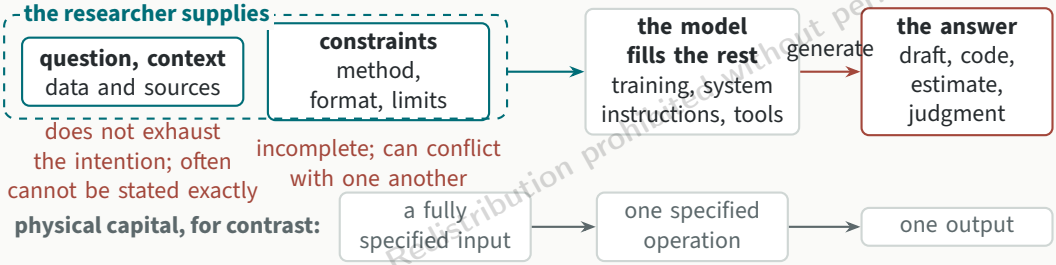
# Unspecified Goals: Learning the Rater, Not the Facts




- **Sycophancy**: the model learns what the rater approves, the rater's biases included, rather than what is true.
- Sharma et al. (2024): feedback warms when the user likes the text; correct answers are dropped when the user objects; preference data reward both.
- Prendergast (1993): a worker judged by the boss's own opinion has reason to repeat it; the yes-man is made by the rule.
- **Specifying and verifying are distinct problems; either way the model can move away from the intent.**

© Zhigang Feng Redistribution prohibited without permission

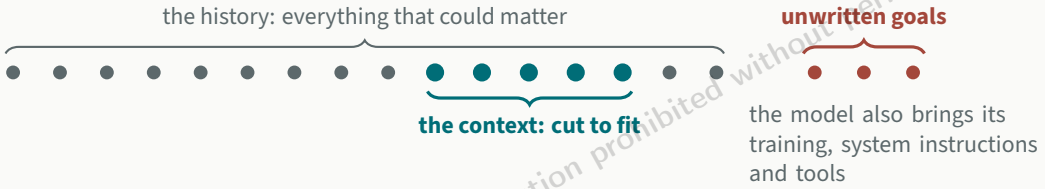
# How a Researcher Works with AI




© Zhiyang Peng
Redistribution prohibited without permission

- Every query hands over a question, a context and constraints; the context cannot exhaust what we want, much of which cannot be stated precisely, and the constraints are incomplete and can conflict.
- **Physical capital takes a fully specified input; producing cognition does not: delegation under an incomplete specification.**

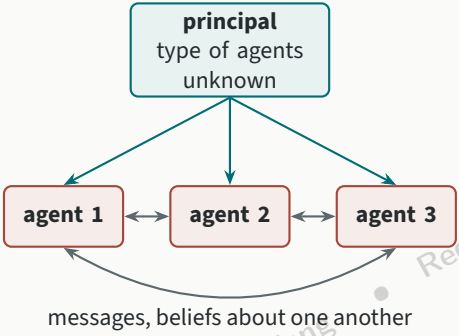
# The Context Is Part of an Incomplete Contract




© Zhiqiang Feng. Redistribution prohibited without permission

- Users supply instructions, materials and context; behaviour also reflects training, system instructions and tools: an *incomplete* delegation arrangement.
- **Compression**: context is finite, unevenly used, written in limited time. Like choosing the **state** of a recursive problem: omit what matters, solve another problem.
- **Tacit expectations**: the standard of evidence, how to reconcile sources. People and models both infer them, and can infer differently.
- **Where the context is silent, training, system instructions and tools fill the gap.**

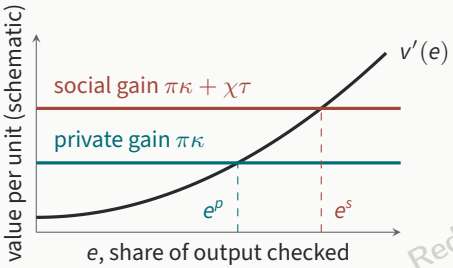
# Units of the Factor Act as Agents




- A programmed system follows specified actions; an agent pursues a goal learned from a formula or a rater, so it is harder to anticipate.
- Bergemann, Koh & Morris (2026): agents whose preferences and capabilities deployers are unsure of.
- Pricing algorithms and the 2026 incidents of Deck 1.1: unintended coordination, a possible failure mode, not its frequency.
- **Owning the model does not remove delegation, monitoring and control problems.**

© Zhigang Feng
Redistribution prohibited without permission

# When Checking Has an External Benefit




$$\text{private:} \quad v'(e^p) = \pi\kappa$$
$$\text{social:} \quad v'(e^s) = \pi\kappa + \chi\tau$$

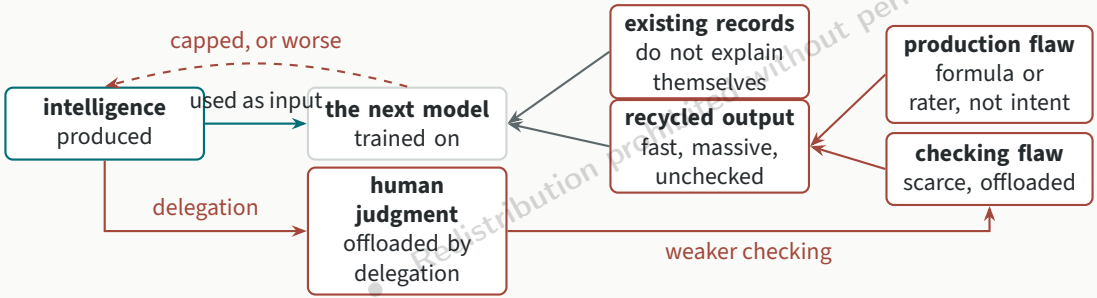
- Benchmark: unpaid for improving a shared corpus, adopters check too little.
- Corrections must *reach* training: check in-house, ship the original, and $\chi = 0$.
- Levers: rights to corrected data, provenance, feedback contracts, pay for evaluation.

$e$ share of machine output checked; $v(e)$ checking cost per unit; $\pi$ error probability; $\kappa$ loss from an unchecked error; $\tau$ value of the data quality one more check preserves; $\chi$ share of corrections that reach training.

- **The question is which arrangement sustains valuable checking with broad access, not whether more checking is always better.**

© Zhiqiang Feng
Redistribution prohibited without permission

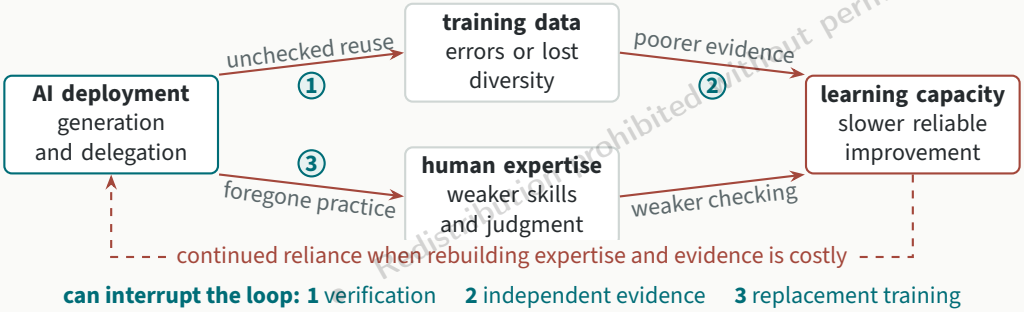
# Putting the Pieces Together




© Zhiyuan Feng · Redistribution prohibited without permission

- Trained on records that explain nothing by themselves and on recycled output, flawed in production (formula or rater, not intent) and in checking (scarce, offloaded).
- **Delegation also offloads human judgment, so the two flaws feed each other: one loop, two channels.**

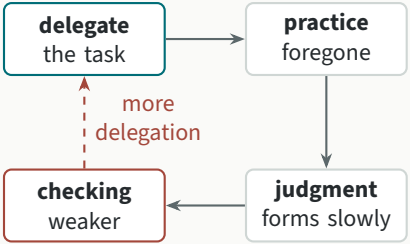
# The Middle-Intelligence Trap




© Liyang Feng
Redistribution prohibited without permission

- *Middle*: useful enough to invite delegation, too unreliable to replace the independent evaluation improvement needs (cf. the middle-income trap; Gill & Kharas 2007).
- Delegation also offloads human judgment: weaker checking lets more error through.
- **A hypothesis: two conditional channels, each of which can be interrupted.**

# Delegation Offloads Human Cognition




the individual channel of the trap: each turn of the loop removes a checker the collective channel needs

- **Cognitive offloading**: using a tool in place of one's own memory or reasoning. It frees time for other work, so its net effect is ambiguous (Risko & Gilbert 2016).
- Judgment is learned by doing the work: drafting, checking, failing. Work that is delegated gives no practice.
- The evaluators of the next generation are trained by today's junior tasks; if those are delegated, who learns to check?
- **Offloading links the individual channel to the collective one: weaker checking lets more recycled error through.**

© Zhigang Feng
Redistribution prohibited without permission

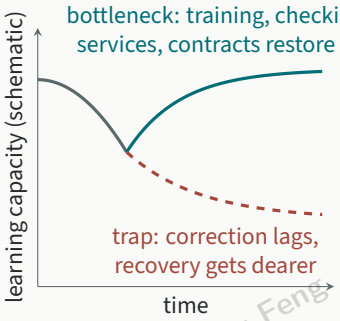
# The Two Channels: What the Evidence Shows

| collective channel: training data | individual channel: human expertise |
|---|---|
| **can hurt**: unchecked reuse carries errors into training; recycled output narrows coverage | **can hurt**: delegation removes the practice through which judgment forms |
| **can help**: externally evaluated output adds information; synthetic data need not harm | **can help**: offloading frees time for other learning; net effect ambiguous (Risko & Gilbert 2016) |

- Scientists: 49% say AI steers them toward safer projects, 28% toward riskier ones (Codreanu et al. 2026): perceptions, not measured novelty.
- Heavier AI reliance goes with weaker critical thinking (Gerlich 2025): a correlation.
- Relative employment at ages 22–25 fell in the most exposed occupations (Brynjolfsson, Chandar & Chen 2025): if junior tasks train evaluators, who trains the next?
- **Meta-explanation, questioning and cross-validating what is generated, may become more valuable.**

© Zhiwei Fan · Redistribution prohibited without permission

# From Bottleneck to Trap: Why Correction Might Lag




- A bottleneck invites investment; a trap needs reasons why that response stays inadequate.
- Dispersed benefits can weaken the incentive to keep shared evidence; once sources blur, new evidence may need costly measurement.
- Skills form slowly: a firm can hire evaluators today, with little reason to fund training that benefits future employers.
- **Errors or lost practice are not enough: a trap needs correction to fail over the relevant horizon.**

© Zhigang Feng
Redistribution prohibited without permission

# Escaping the Trap

| **cognitive reserves** | **strategic friction** | **evaluation** | **economics education** |
|---|---|---|---|
| unassisted practice: which skills depreciate, and how fast? | engagement at judgment points, traded against convenience | schools, journals and firms that also reward reasoning and meta-explanation | graduates prepared for the human–AI division of labour |

© Lisa-Jane Feuß · non-distribution prohibited without permission

- Four possible responses, each a question economists can evaluate.
- Friction: where to require engagement, and when do its private and external benefits justify the cost?
- The underprovision result is a potential rationale for intervening where corrections reach a shared corpus. It does not show these responses work, or rank them against standards, liability or provenance rules.
- **Each must be weighed against its costs and its effects on access, innovation and learning.**

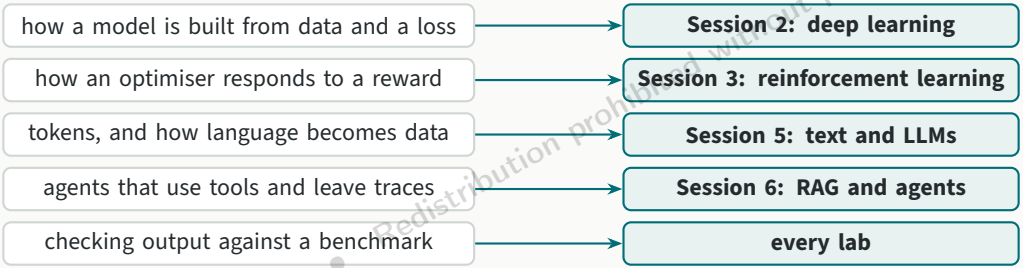
# An Invitation: Open Questions

| | |
|---|---|
| **a produced factor** | which cognitive services can be supplied apart from scarce human time, and how is useful output measured beyond the token? |
| **deployment in a job** | when does AI on an occupation's core tasks substitute for its workers, and when does it expand demand for the rest of the job? |
| **checking and directing** | how much checking is optimal, who captures its return, and how should contracts allocate losses? |
| **the feedback** | how fast do skills depreciate under delegation, who pays for apprenticeship, and when does a bottleneck persist? |

© Shuguang Wang
Restricted for profit use without permission

- Three agendas: *measurement* of useful cognitive services and their deployment (Deck 1.1); *theory* of growth and market structure when data quality is endogenous; *institutions* that sustain verification, independent evidence and expertise.
- **The evidence and the externality result are first steps; the rest is open: an invitation.**

# Why This Course Opens the Box




© Zhiyang Feng · Redistribution prohibited without permission

- To study these phenomena, and to use the tools well, you need to know how the machine works.
- **Understand how it works, to understand what it does.** Next, Deck 1.3: what AI is.

Session 1.3

# What Is AI? Foundations

**A working definition, seventy-five years of history, why now, and what an agent is**




Redistribution prohibited without permission

Zhigang Feng

© Zhigang Feng

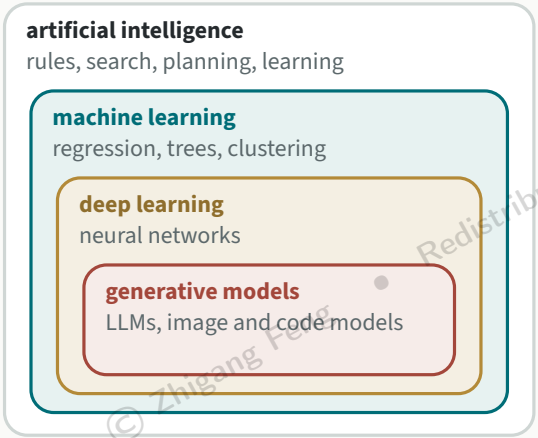
# What Do We Mean by AI?

- A working definition: methods that let machines do tasks that normally need human intelligence.
- **AI is not machine learning.** Machine learning, a subfield, studies systems that improve with experience. A rule-based scheduler, or a chess program that searches moves, is AI with no learning.
- The **rational-agent** view: a system that perceives, persists, and acts for the best expected outcome. Economics has studied that object for two centuries.
- The economist's test is not "does it understand?" but what it lets you **measure, predict, identify, compute**.
- People are strong at *abduction*, the best explanation from little evidence; machines at *induction*, patterns from many examples. The boundary is moving.
- **Whether a machine thinks is philosophy. What a usable proxy or a solvable model is worth is economics.**

© Zhiguo Feng · Reproduction prohibited without permission

Source: Russell & Norvig (2021); Miao & Feng, *AI for Economists*, §1.1; Feng, working paper (2026), §2.

# The Landscape




- **AI**: systems that perform tasks associated with intelligence.
- **Machine learning**: performance improves with data or experience.
- **Deep learning**: layered representations learned by neural networks.
- **Generative models**: produce new content given a context.
- **An agent is not a fifth ring.** It is a *system* built around a model: a loop with tools, memory and permissions.

© Zhigang Feng Redistribution prohibited without Permission

# One Question, Three Eras

**Forecast next quarter's GDP growth.** The question never changed. The answer did, three times.

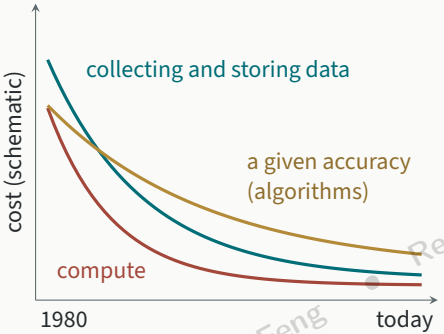
| Era (approx.) | Method | Data | What is assumed |
|---|---|---|---|
| 1980–2000 | autoregressions, VARs | a handful of series | linear, stationary dynamics |
| 2000–2015 | factor models (Stock–Watson: 215 series), shrinkage, nowcasting | hundreds of series | a few common factors, or small coefficients |
| 2015– | forests, boosting, neural networks | series plus text and images | little about functional form |

(c) Zhigang Feng Redistribution prohibited without permission

- **Each era added tools without retiring the old ones. Factor models and Bayesian VARs remain the benchmark that machine learning must beat.**

Source: Sims (1980); Stock & Watson (2002); Giannone, Reichlin & Small (2008); Goulet Coulombe et al. (2022).

# What Drove Each Transition

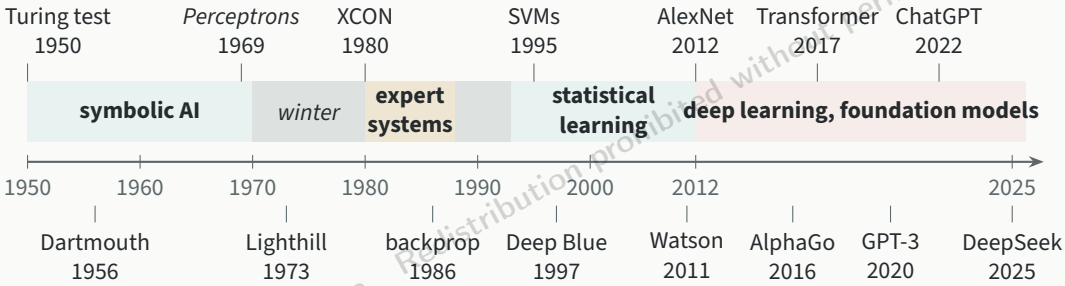



© Zhigang Feng · Redistribution prohibited without permission

- The ideas often existed already. They paid off when three costs fell together: **data**, **compute**, and the cost of a given accuracy.
- They are complements: one alone changes little.
- Varian: larger data sets call for new tools, such as variable selection, out-of-sample validation and trees.
- **When data and computing get cheaper, the best method changes.**

Source: Miao & Feng, *AI for Economists*, §1.2; Varian, "Big data: new tricks for econometrics", *JEP* (2014). Curves are schematic.

# Seventy-Five Years in One Picture




© Zhiguo Feng
Redistribution prohibited without permission

- Three waves, separated by two “AI winters” (grey): funding collapses when an expectations bubble bursts.
- **Each winter was also the start of the next accumulation.** The time axis stretches after 2012.

Source: Miao & Feng, *AI for Economists*, §1.3.

# Wave 1: Symbolic AI and the First Winter

- **1950.** Turing replaces “can machines think?” with a test of behaviour.
- **1956.** The Dartmouth workshop names the field. Its 1955 proposal (McCarthy, Minsky, Rochester, Shannon): every feature of intelligence “can in principle be so precisely described that a machine can be made to simulate it”.
- **1965.** Herbert Simon: within twenty years machines will do any work a person can do.
- **1958–69.** Rosenblatt’s perceptron learns from examples. Minsky and Papert prove that one layer cannot learn XOR, and neural networks are shelved.
- **1973.** The Lighthill report: no part of the field had produced “the major impact that was then promised”. Britain cuts university AI funding; in the US, DARPA narrows support to mission-oriented work.
- **An expectations bubble, read as an economist: early wins extrapolated linearly, investment ahead of delivery, then a correction.**

© Shuguang Feng. Redistribution prohibited without permission

Source: Turing (1950); Rosenblatt (1958); Minsky & Papert (1969); Lighthill (1973); Miao & Feng, *AI for Economists*, §1.3.

# Expert Systems and the Second Winter

- A new route: write a specialist's judgment down as IF–THEN rules, and let a program apply them.
- Real commercial success: DEC's XCON was saving, by 1986 and by DEC's own estimate, about $40 million a year configuring computer orders.
- Japan announced its Fifth Generation project (1981); by 1985 corporations were spending over $1 billion a year on AI, mostly in-house, and industry sales reached about $2 billion by 1988.
- The flaw was the **knowledge bottleneck**. Much of what experts know is tacit. They can do it, and cannot say how.
- It is Hayek's knowledge problem: knowledge is dispersed and hard to articulate, so centralising it in explicit rules fails once the system is complex enough.
- **The lesson of both winters: stop encoding knowledge; let the machine find patterns in examples.**

(C) Zhaguan Ng Redistribution not permitted without permission

Source: Miao & Feng, *AI for Economists*, §1.3; Hayek, "The use of knowledge in society", *AER* (1945).

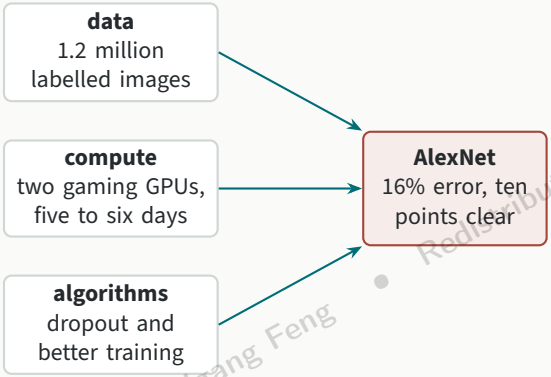
## Wave 2: Statistical Learning

- **1986.** Rumelhart, Hinton and Williams make backpropagation practical: the chain rule carries the output error back through the layers, so deep networks can be trained in principle.
- In practice, compute and data were still too scarce for deep networks.
- **1990s.** Support vector machines give classification an elegant theory. LeCun's convolutional network reads handwritten digits (1998).
- The strategy: "small targets, big data". Give up general intelligence; win one measurable task at a time, such as spam, clicks, digits.
- **Deep Blue (1997) searched up to 200 million positions a second. AlphaGo (2016) learned. That contrast is the story of the next slide.**

© Zhiyang Feng · Redistribution prohibited without permission

Source: Rumelhart, Hinton & Williams, *Nature* (1986); LeCun et al. (1998); Silver et al., *Nature* (2016).

# Wave 3: The 2012 ImageNet Moment

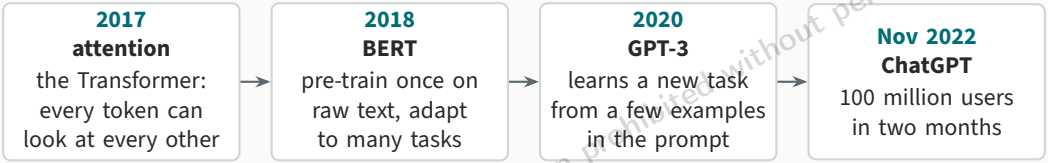



- The architecture was not new: convolutional networks date from 1989.
- New was that three complements arrived *together*. Remove any one and nothing happens.
- **Complementary innovation**: each input alone has low marginal value.
- **That is why breakthroughs are lumpy. They wait for the last complement.**

Source: Krizhevsky, Sutskever & Hinton, NIPS (2012); Miao & Feng, *AI for Economists*, §1.3.

© Zhigang Feng
Redistribution prohibited without permission

# Transformers and Foundation Models

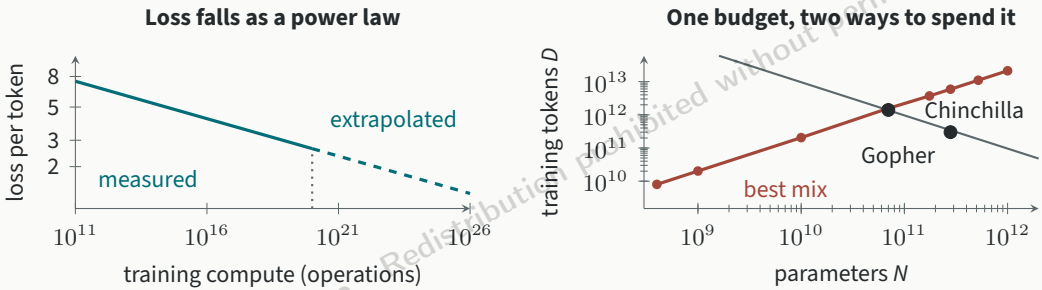



© Zhirang Feng Redistribution prohibited without permission

- A **token** is a sub-word unit of text: what these models read, write and are priced in.
- **Foundation model**: one large model trained on broad data, then adapted to many downstream tasks.
- The economics changes with it: a huge fixed cost of training, a small marginal cost of use, and one model serving many markets.
- **One architecture now handles text, code, images, audio and protein sequences.**

Source: Vaswani et al. (2017); Devlin et al. (2018); Brown et al. (2020); Miao & Feng, *AI for Economists*, §1.3.

# Why Now (1): Scaling Laws

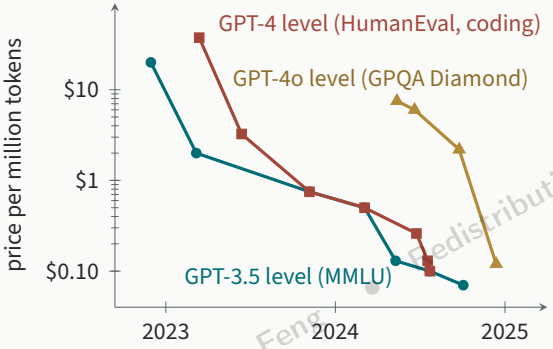



Redistribution prohibited without permission

- Ten times the compute cuts the loss by a tenth. On one budget (grey, an isocost line), Chinchilla, smaller but fed more data, beat Gopher.
- **A regularity, not a theorem. Predictable returns are what justify a trillion dollars of capex.**

Source: Kaplan et al. (2020); Hoffmann et al. (2022); Miao, Yuan & Feng, *Introduction to AI Economics*, ch. 2.

# Why Now (2): Efficiency and Price




- The cheapest model matching a *fixed* capability: about 280 times cheaper at GPT-3.5 level in under two years.
- Much of it is algorithmic efficiency, not scale.
- DeepSeek-R1 (2025): o1-level reasoning; its final training runs cost a reported $6M (research excluded); GPT-4, reportedly over $100M.
- **Cheaper entry means faster diffusion.**

© Zhigang Feng · Redistribution prohibited without permission

Source: Stanford HAI, *AI Index Report 2025*, Fig. 1.3.22 (Epoch AI; 3:1 blended prices); DeepSeek-V3 and R1 reports; GPT-4 as reported by Altman.

# 2023–2026: From a Chat Box to Agent Teams




© Zhigang Feng Redistribution prohibited without permission

- Alongside: *reasoning models* spend extra compute at answer time to think before they reply.
- **What changed research practice is where the loop runs and what it can touch. Each step made the work cheaper; none made checking it cheaper.**

Source: Dates checked in September 2026 against vendor announcements; Session 6 explains the protocol.

## Three Ways to Learn

| Paradigm | Feedback | What is learned | In economics | Session |
|---|---|---|---|---|
| Supervised | labelled pairs $(x, y)$ | a mapping $x \mapsto y$ | default risk, a GDP forecast | 2, 5 |
| Unsupervised | observations $x$ only | clusters, factors, topics | grouping business cycles | 5 |
| Reinforcement | a reward along a path | a state-dependent policy | a consumption–saving rule | 3 |

- All three pick a function from a *hypothesis space* by optimising an objective. They differ in where the feedback comes from.
- **OLS, factor models and dynamic programming are the versions you already know.**

Source: Miao & Feng, *AI for Economists*, ch. 2.

© Zhiyang Feng · Reproduction prohibited without permission

# Supervised Learning as Empirical Risk Minimisation

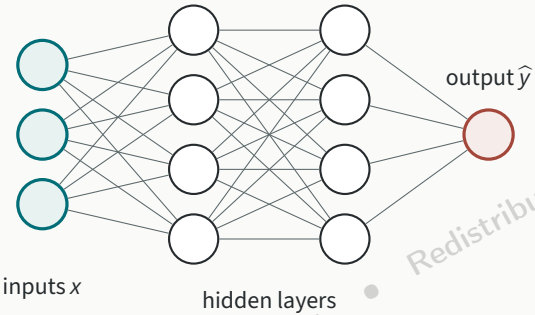
$$\widehat{f} \;=\; \arg\min_{f \in \mathcal{F}} \; \frac{1}{n} \sum_{i=1}^{n} L\big(y_i,\, f(x_i)\big) \;+\; \lambda\, R(f)$$

- Four choices define the problem: the **hypothesis space** $\mathcal{F}$, the **loss** $L$, the **regulariser** $R$ with weight $\lambda$, and a **held-out rule** for judging the result.
- OLS is the special case: linear $\mathcal{F}$, squared loss, no regulariser. Lasso adds $R(f) = \|\beta\|_1$.
- A flexible $\mathcal{F}$ fits the noise in the sample. That is **overfitting**, and the held-out rule is what detects it.
- Uses in economics: nowcasts, classifying firms and households, risk and demand prediction, and nuisance functions inside causal estimators.
- **Session 2 builds on this slide: a neural network is one choice of $\mathcal{F}$.**

Source: Miao & Feng, *AI for Economists*, ch. 2.

© Zhiguang Feng
Redistribution prohibited without permission

# What a Neural Network Is




$$h^{(\ell)} = \sigma\big(W^{(\ell)} h^{(\ell-1)} + b^{(\ell)}\big)$$

- Each layer: an affine map, then a simple nonlinearity $\sigma$.
- **Width** supplies basis functions; **depth** composes them.
- The **weights** $W, b$ are the parameters: thousands in today's lab, billions in an LLM.
- **A flexible function approximator. Session 2 builds it from one neuron.**

Source: Miao & Feng, *AI for Economists*, ch. 2; LeCun, Bengio & Hinton, *Nature* (2015).

© Zhigang Feng
Redistribution prohibited without permission

# How a Network Learns




© Thiago Ferro. Not to be distributed or reproduced without permission

- **Gradient descent**: move every weight a small step in the direction that lowers the loss.
- **Backpropagation** is the chain rule, organised so that all the derivatives cost about as much as one forward pass.
- Stochastic: each step uses a random batch, so training is a noisy walk downhill.
- **Backpropagation computes derivatives. It does not choose the loss, the data, or the claim those choices can support.**

Source: Rumelhart, Hinton & Williams (1986); Miao & Feng, *AI for Economists*, ch. 2.

# From Words to Vectors to Attention

| Representation | What it keeps | What it loses, or adds |
|---|---|---|
| Bag of words | word counts | loses order and context |
| Static embedding | similar words sit close together | one vector per word, whatever the sentence |
| Contextual embedding | meaning given the surrounding words | "rate" differs in "cut the rate" and "rate of return" |
| Transformer | learned attention across the whole sequence | long-range context, at a cost quadratic in length |

© Zhiang Feng · Redistribution prohibited without permission

- **Embedding**: a map from words to vectors. Text becomes a regressor, but an *estimated* one: variables built from it carry the model's errors.
- **Attention routes information inside the text it is given. It does not retrieve truth.**

Source: Miao & Feng, *AI for Economists*, §1.5 and ch. 13; Session 5 owns the mechanics.

# An LLM Predicts the Next Token

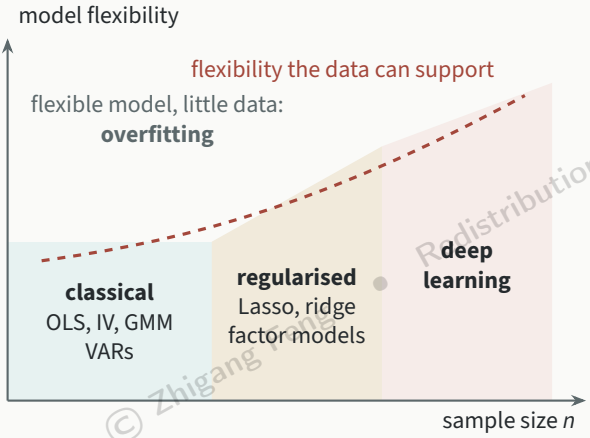
$$P_\theta(t_1, \ldots, t_T) = \prod_{i=1}^{T} P_\theta\big(t_i \mid t_{<i}\big), \qquad \mathcal{L}(\theta) = -\sum_i \log P_\theta\big(t_i \mid t_{<i}\big).$$

- A **token** is a sub-word unit of text. Training is next-token prediction; summarising, coding and dialogue come from conditioning that one objective on instructions and context.
- Output is **probabilistic**, and the objective rewards **plausibility**: sounding right and being right are different targets.
- Knowledge is bounded by the **training cut-off** and by the **context window**, the text the model can see at once.
- **Temperature** changes how the next token is sampled. It does not change what is true.
- **Session 1's lab lets you turn that dial and watch fluency and reliability come apart.**

© Zhiguang Feng · Redistribution prohibited without permission

Source: Miao & Feng, *AI for Economists*, ch. 13.

# Strong Assumptions or Flexible Data




© Zhigang Feng
Redistribution prohibited without permission

- Econometrics trades *assumptions* for efficiency: with about 300 quarters of post-war GDP there is little choice.
- Machine learning trades *data* for flexibility.
- "Weak assumptions" are not none. They move into the architecture, where they are harder to see.
- **Complements: theory fixes what must stay invariant; flexible methods estimate the rest.**

Source: Miao & Feng, *AI for Economists*, §1.4; Athey & Imbens, *Annual Review of Economics* (2019).

# What More Data Cannot Fix

| | small data | big data |
|---|---|---|
| **the target is a prediction** | a few-variable regression<br>*limit: bias from a rigid form* | machine-learning predictors<br>*gain: approximation bias falls with n* |
| **the target is a causal effect** | IV, RDD, difference-in-differences<br>*limit: low power* | causal ML (double ML)<br>*trap: unobserved confounding, now very precise* |

© Zhigang Feng Redistribution prohibited without permission

- Endogenous $x$: $\operatorname{plim}\hat{\beta} = \beta + \operatorname{Cov}(x,u)/\operatorname{Var}(x)$. More data narrows the interval, not the gap. *Identification* bias does not depend on $n$.
- **The Lucas critique is a distribution shift the policy itself causes: people's decision rules change with the rule, so a model fitted under the old rule fails.**

Source: Miao & Feng, *AI for Economists*, §§1.5–1.6; Lucas (1976).

# What AI Is Good At, and What It Is Not

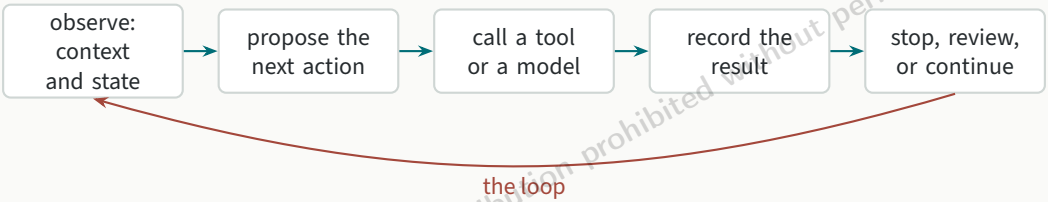
| Task | Where AI is strong | The core limit | In economics |
|---|---|---|---|
| Classify, predict | many inputs, large samples | distribution shift | credit scoring |
| Read text | meaning, tone, topics | confident invention | central-bank communication |
| Read images | features, objects | needs labels | activity from satellites |
| Causal inference | *a helper only* | cannot settle identification | nuisance functions in double ML |
| Policy search | large state spaces | the reward is hard to design | simulating monetary rules |

© Lihang Feng • Redistribution prohibited without permission

- **AI widens what economists can measure, predict and compute. It does not change the logic of identification.**

Source: Miao & Feng, *AI for Economists*, §1.6.

# From Generation to Agency

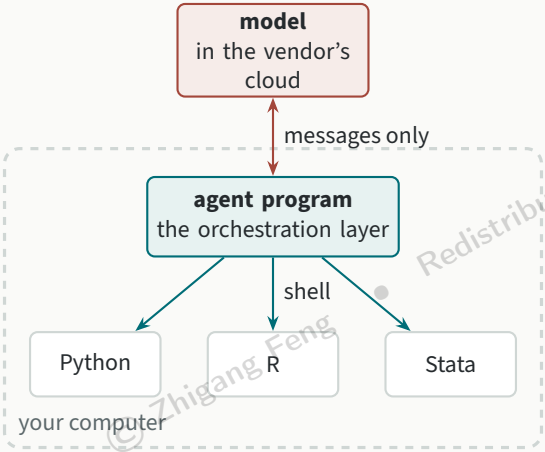



© Zhirang Feng · Redistribution prohibited without permission

- A language model, by itself, returns text and stops. It delivers advice.
- An **agent** is a *system*: a model, instructions, tools, memory, permissions, and a stopping rule, run in a loop. It delivers actions.
- Four marks: it uses tools, acts with autonomy, iterates, and persists across steps.
- **Change the wrapper and the behaviour changes, with the weights held fixed.**

Source: Miao & Feng, *AI for Economists*, ch. 15; Yao et al., "ReAct" (2023). Session 6 owns the architecture.

# How an Agent Works on Your Machine




Source: Miao & Feng, *AI for Economists*, ch. 15.

- The agent program runs locally. It does no econometrics itself.
- It asks the model "what next?", then runs that step through the shell in the Python, R or Stata you already have.
- Files stay on your machine. Only structured messages cross to the model.
- **You do not give up Stata. You gain a layer that types commands, reads errors, and tries again.**

© Zhigang Feng
Redistribution prohibited without permission

# Cheat Sheet

| Claude Code | What it does, and why it earns a place |
|---|---|
| `/init` | writes a first standing-instructions file from the repository — the brief re-read at every start |
| `/plan` | enables plan mode: it explores and drafts, and changes nothing until you approve |
| `/model` | switches model mid-session, context kept: a derivation and a rename do not deserve the same engine |
| `/effort` | sets the effort level for the session — the largest quality-against-cost dial you hold |
| `/context` | draws context use as a grid: the invisible constraint, made visible |
| `/compact` | summarises the session so far — one you choose beats one that fires mid-derivation |
| `/rewind` | restores the conversation, or the files, to a checkpoint (Escape twice) |

© Zhigang Feng · Redistribution prohibited without permission

**Four dials: what it reads, how hard it thinks, how much it may do unasked, what it costs.**

Source: Claude Code's command help, course machine, 20 September 2026. Session 6 returns to each command (Deck 6.1).

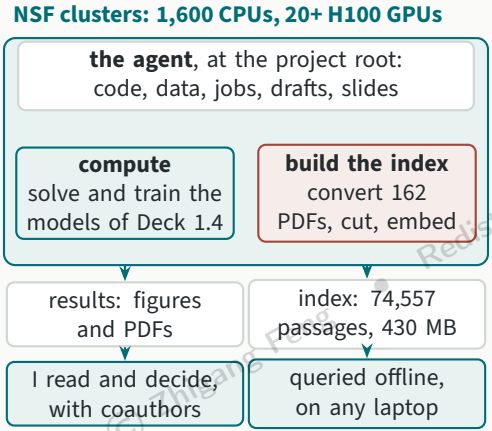
# How Agents Help Economic Research

| Task | What the agent does | The check you owe |
|---|---|---|
| Literature mapping | expands and triages a corpus | coverage; every source matches its claim |
| Replication audits | reruns code, compares tables | a clean run from the raw data |
| Data construction | finds, cleans and merges sources | spot checks against official totals |
| Model solution | approximates policies and values | residuals, and known special cases |
| Theory to code | pseudocode, tests, code variants | validation gates set before the run |
| Building on a literature | retrieves models by their conditions, not their wording | the conditions of each result still hold |

© Shugang Feng · Redistribution prohibited without permission

- The last two rows are this instructor's designs (a theory-to-code loop; condition-aware retrieval, Feng et al. 2026). Designs only; Session 6 shows them at work.
- **Delegate first the tasks whose errors are cheap to see.**

# How I Work: A Personal RAG, and Everything on a Cluster



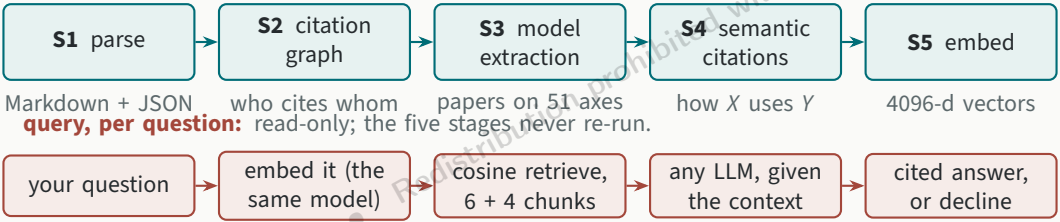

- **RAG**: the agent answers from my own library, citing the page, or says the evidence is weak.
- Build once, on a GPU: a vision model reads each PDF page (*convert*), the text is cut on its headings (*cut*), each passage becomes 1,024 numbers (*embed*).
- The same allocations solve and train Deck 1.4's models.
- **Build where the data and the GPUs are; read and decide anywhere.**

Source: The instructor's retrieval system, read 12 October 2026, and project record; allocations through NSF grants. Session 6 builds both (Decks 6.1 and 6.2).

© Thomas Reed. Redistribution prohibited without permission

# GRAM: A Retrieval System Built for One Literature

**build once per corpus.** Three LLMs in three roles: *reading* (page images to Markdown), *reasoning* (text to structured knowledge), *embedding* (text and the question to one 4096-d space).

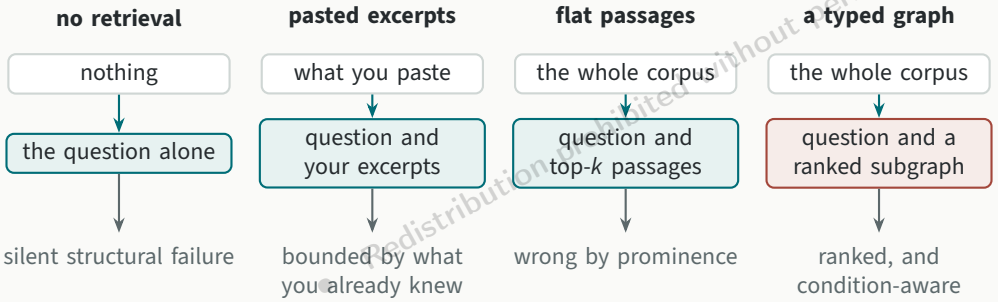



- **GRAM** (Generative Reasoning and Augmented Modeling): retrieval for one literature, quantitative macroeconomics; 133 papers in the pilot (Feng, Miao, Xie, Zhong & Zhou 2026, in progress).
- **Retrieve models by their conditions, not their wording.**

Source: Feng, Miao, Xie, Zhong & Zhou (2026), work in progress.

© Zhigang Feng Redistribution prohibited without permission

# Four Ways to Give a Model Context, on One Question

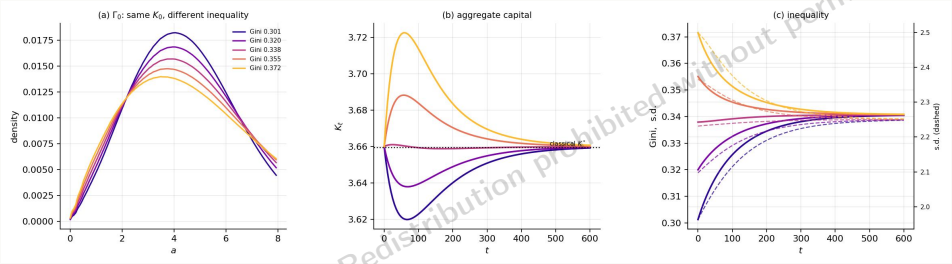



© Zhigang Feng · Redistribution prohibited without permission

- The generator is the same in all four. Only what reaches it changes.
- **These name each mechanism's failure mode on one kind of question. They are not a ranking of architectures.**

Source: Feng, Miao, Xie, Zhong & Zhou (2026), work in progress.

# Research in Practice: Optimal Taxation With Heterogeneous Agents




Redistribution prohibited without permission

© Zhuang Feng

- The benchmark step of the project: five Aiyagari economies with the *same* $K_0 = 3.660$ and Gini 0.301 to 0.372 all reach the classical $K^* = 3.6595$.
- A network trained on $\Gamma_{t+1} = \Phi(\Gamma_t)$ makes convergence a result, not an assumption; Session 4 adds the tax.
- **Same $K_0$, yet the distributions move for hundreds of periods. $K$ alone misses it.**

Source: Feng, Han, Sargent & Zhu (2026), work in progress. Terminal $K$ agrees to $3.5 \times 10^{-8}$.

## What Stays Human

**You own**

- the question, and why it matters;
- the identification strategy;
- the state variables and the equilibrium concept;
- the benchmark, chosen before the answer is known;
- the reading of a failure;
- the responsibility.

**The agent speeds up**

- search, reading, and first drafts;
- data cleaning and merging;
- code, tests, and variants;
- runs, logs, and figures;
- checks you have specified.

- **The division is not by difficulty. It is by who can be held to account.**

© Zhiguo Feng · Redistribution prohibited without permission

# Six Sessions: Where Each Piece Returns

| Session | Object | What changes |
|---|---|---|
| 1 | Orientation | AI as tool, as object, and as an agent with incentives |
| 2 | Deep learning | Functions become trainable; equations become losses |
| 3 | Reinforcement learning | Sequential decisions are learned from interaction |
| 4 | Heterogeneous agents | Distributions and equilibrium become state variables |
| 5 | Text and LLMs | Language becomes data, measurement, and generation |
| 6 | RAG and agents | Retrieval, tools, traces, and governance in one workflow |

© Zhiguang Feng · Unauthorized distribution prohibited without permission

- Networks and training return in Session 2; learning from rewards in 3; distributions in 4; tokens and attention in 5; agents and retrieval in 6.
- **Next, Deck 1.4: the economic models these methods will solve.**

Session 1.4

# Workhorse Macroeconomic Models

**The models this course solves, the object each one asks for, and where grids stop working**




© Zhigang Feng
Redistribution prohibited without permission

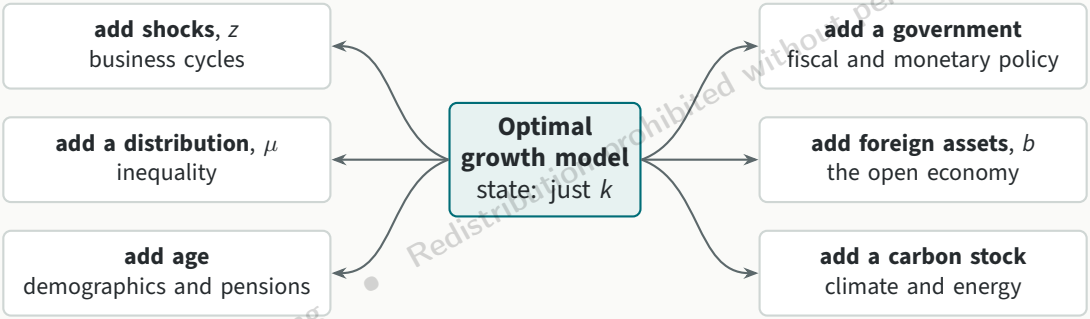
Zhigang Feng

# Why This Deck, and Who It Is For

- The course is built for macroeconomists. The methods carry over to dynamic games in industrial organisation, asset pricing, and labour search.
- What AI will do for us: **(1)** solve the workhorse models efficiently and at scale; **(2)** turn unstructured data into measurement; **(3)** help us understand and write models, design algorithms, and implement code.
- **(4)** Verification, judgment and responsibility stay with the researcher.
- A word of warning: in this deck an **agent** is a household or a firm inside a model. That is the word's third meaning today, after the AI agent and the agent of contract theory.
- **For each model, ask: what is the state, what object must be computed, and what check would expose a wrong answer?**

© Zhiguo Feng · Redistribution prohibited without permission

# One Model, Many Directions




© Zhiyang Feng Redistribution prohibited without permission

- The trunk is one planner choosing consumption and capital. Each branch is a field of macroeconomics.
- **Every branch adds state variables, and more state means harder to compute.**

# The Environment

$$\max_{\{c_t,\, k_{t+1}\}} \sum_{t=0}^{\infty} \beta^t \log c_t \qquad \text{subject to} \qquad c_t + k_{t+1} = A k_t^{\alpha}, \qquad k_0 \text{ given.}$$

- One good, one decision-maker. Output $Ak^{\alpha}$ is either eaten ($c$) or saved as tomorrow's capital ($k'$).
- $A$ is **total factor productivity** (TFP), $\alpha$ the capital share, $\beta$ the discount factor.
- Log utility and full depreciation are chosen on purpose: this version has a closed-form solution.
- **State**: what you must know today to decide ($k$). **Control**: what you choose ($k'$).
- **One calibration wherever this model returns (today's lab, Session 2): $\alpha = 0.33$, $\beta = 0.96$, $A = 1$.**

Source: Brock & Mirman, "Optimal economic growth and uncertainty: the discounted case", *Journal of Economic Theory* (1972).

© Zhecheng Tang · Redistribution prohibited without permission

# Why the Bellman Equation Can Be Trusted

$$V(k) \;=\; \max_{0<k'<Ak^{\alpha}} \Big\{ \log\!\big(Ak^{\alpha} - k'\big) + \beta\, V(k') \Big\} \;\equiv\; (TV)(k)$$

| Result | What it guarantees |
|---|---|
| Principle of optimality | the sequence problem and the Bellman equation share a solution |
| Blackwell's conditions | monotonicity and discounting make $T$ a contraction, modulus $\beta$ |
| Contraction mapping | one fixed point $V$; $V_{n+1} = TV_n$ reaches it from any $V_0$ |
| Theorem of the maximum | $V$ is continuous, and an optimal choice exists at every $k$ |
| Concavity | $V$ strictly concave, so the policy $k' = g(k)$ is a function |
| Benveniste–Scheinkman | $V$ differentiable; the envelope condition gives the Euler equation |

- **Two descriptions of one solution, a fixed point of $T$ and the Euler equation: two ways to compute it.**

Source: Bellman (1957); Blackwell (1965); Benveniste & Scheinkman, *Econometrica* (1979); Stokey, Lucas & Prescott (1989), ch. 4.

© Zhenglong Feng. Redistribution prohibited without permission

# Solving It, Approach 1: Iterate on the Value Function

## Value function iteration

1. Grid $k_1 < \cdots < k_N$; guess $V_0 \equiv 0$; tolerance $\varepsilon$.
2. For every $k_i$, the Bellman update:

$$V_{n+1}(k_i) = \max_{k_j < A k_i^{\alpha}} \left\{ \log(A k_i^{\alpha} - k_j) + \beta V_n(k_j) \right\},$$

   and record the maximiser $g_{n+1}(k_i) = k_j$.
3. If $\max_i |V_{n+1}(k_i) - V_n(k_i)| < \varepsilon$, stop; otherwise $n \leftarrow n + 1$, go to 2.
4. Return $V_{n+1}$ and $g_{n+1}$.

- Convergence is guaranteed: the distance to $V$ shrinks by $\beta$ each step; from $V_0 = 0$ it takes 417 steps to reach $10^{-6}$ (step 3's rule stops after about 340).
- The policy settles long before the value. **Policy iteration** (Howard 1960) exploits this.
- **Cost per step: $N \times N$ comparisons. Session 3 replaces the grid with a network.**

Source: Bellman (1957); Howard (1960). Iteration count at $\alpha = 0.33$, $\beta = 0.96$, $A = 1$.

© Zhe-ning Peng. Unauthorized distribution prohibited without permission

# Solving It, Approach 2: Solve the Equilibrium Conditions

## Time iteration on the Euler equation

1. Grid $k_1 < \cdots < k_N$; guess a policy $g_0$, e.g. $g_0(k) = \frac{1}{2} A k^{\alpha}$.
2. For every $k_i$, solve for $k'$:

$$\frac{1}{A k_i^{\alpha} - k'} = \beta \frac{\alpha A k'^{\alpha - 1}}{A k'^{\alpha} - g_n(k')},$$

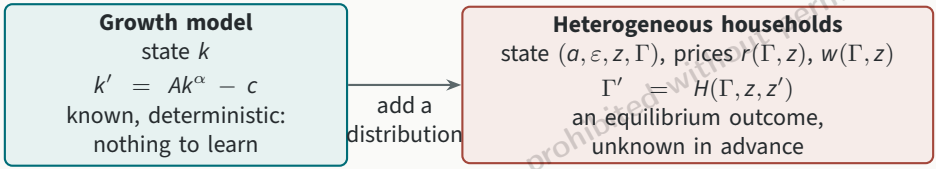
   with $g_n$ interpolated; set $g_{n+1}(k_i) = k'$.
3. Stop once $\max_i |g_{n+1}(k_i) - g_n(k_i)| < \varepsilon$.
4. Check the Euler residual on fresh $k$ off the grid.

Source: Coleman (1990); Judd, *JET* (1992); Carroll, *Economics Letters* (2006).

- The unknown is the policy $g$, never $V$.
- Variants: projection (Judd 1992) sets the residual to zero at nodes; endogenous grids (Carroll 2006) invert the equation, with no root-finding.
- **Euler alone admits explosive paths; transversality rules them out. Session 2 trains a network on the squared residual.**

© Zhiyang Feng Redistribution prohibited without permission

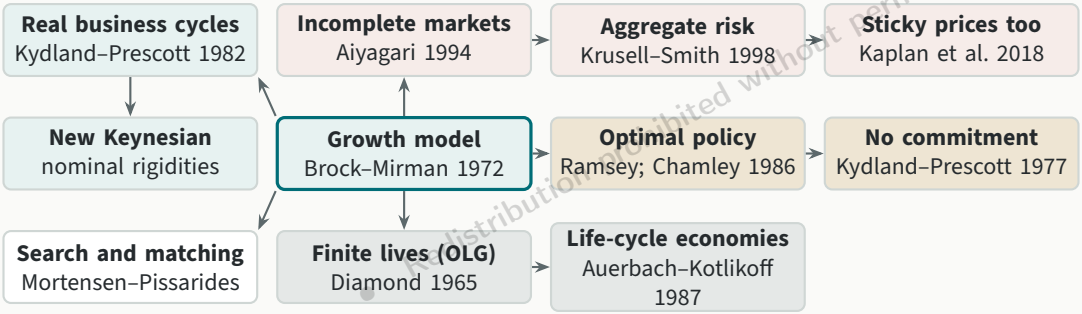
# When the Environment Is Unknown




© Zhiyang Feng
Redistribution prohibited without permission

- In reinforcement-learning terms the **environment** is the transition law and the reward.
- $\Gamma$ is the distribution of households over wealth and productivity. Its law of motion $H$ comes from everyone's choices, so it cannot be written down before the household problem is solved.
- Classical answer: guess $H$ from a few moments (Krusell–Smith, later in this deck).
- Reinforcement learning: learn from simulated transitions $(s, a, r, s')$. $H$ is known to the simulator, never to the learner, and expectations become sample averages.
- **Trivial in the growth model; the central difficulty once a distribution is a state.**

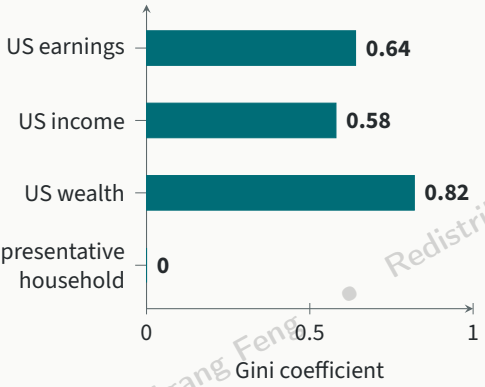
# A Family Tree of Workhorse Models




© Zhigang Feng
No distribution permitted without permission

- Teal: one representative household. Red: heterogeneity. Gold: policy. Grey: the life cycle.
- **Every branch now has a deep-learning frontier.** Sessions 2 to 4 walk the red, gold and grey ones.

# A Puzzle: Wealth Is Far More Unequal Than Income

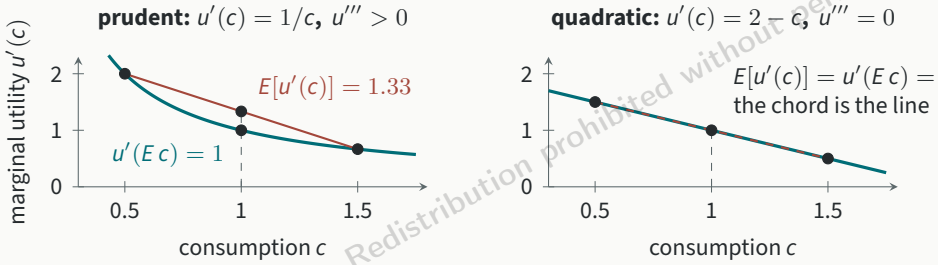



© Zhigang Feng Redistribution prohibited without permission

- **Gini coefficient**: 0 if everyone holds the same, 1 if one household holds everything.
- The top 1% hold about a third of US wealth. A representative household has no distribution at all.
- The textbook incomplete-markets model gets about half of the wealth Gini or less: 0.32–0.41 against 0.82.
- **Every candidate fix (entrepreneurs, bequests, unequal returns) is a new state variable.**

Source: Survey of Consumer Finances 2007, in Díaz-Giménez, Glover & Ríos-Rull, *FRB Minneapolis Quarterly Review* (2011).

# Why Households Self-Insure: Prudence, Not Risk Aversion



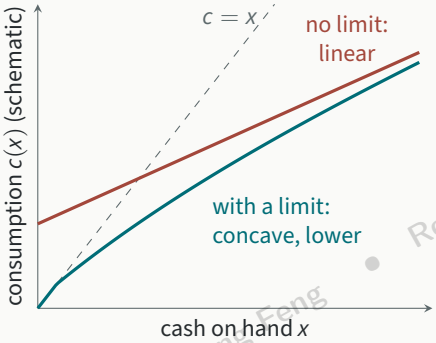

Redistribution prohibited without permission

- Both are risk averse ($u'' < 0$); $c_{t+1}$ is 0.5 or 1.5 with equal odds. In $u'(c_t) = \beta(1+r)E[u'(c_{t+1})]$, risk raises the right side only on the left.
- Left: saving rises. Right: saving is as under certainty (**certainty equivalence**).
- **Precautionary saving needs prudence ($u''' > 0$), not risk aversion alone.**

Source: Leland (1968); Sandmo (1970); Kimball, *Econometrica* (1990).

© Zhiguang Wang

# A Second Motive: The Borrowing Limit




- Quadratic utility, $u''' = 0$. Without a limit, $c(x)$ is linear and risk does not move it.
- With a limit, a bad draw tomorrow cannot be smoothed by borrowing. The household consumes less today, most of all when its cash on hand is low.
- **In Aiyagari both motives operate: prudence and the borrowing limit.**

Source: Zeldes, *Quarterly Journal of Economics* (1989); Carroll & Kimball (2001). Curves are schematic.

© Zhigang Feng
Redistribution prohibited without permission

# Aiyagari: The Household and the Firm

$$v(a,\varepsilon) = \max_{c,\,a'} \left\{ u(c) + \beta \sum_{\varepsilon'} \pi(\varepsilon' \mid \varepsilon)\, v(a',\varepsilon') \right\} \quad \text{s.t.} \quad c + a' = (1+r)a + w\varepsilon, \qquad a' \geq -\underline{a}.$$

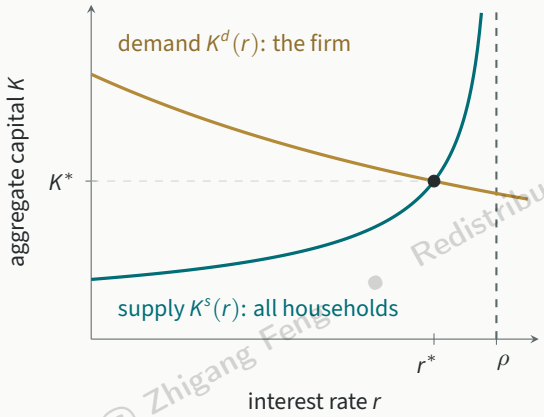
$$r = \alpha A\,(K/L)^{\alpha-1} - \delta, \qquad w = (1-\alpha) A\,(K/L)^{\alpha}.$$

- $a$: assets. $\varepsilon$: labour productivity, a Markov chain no one can insure. $\underline{a}$: the borrowing limit.
- A continuum of households, identical ex ante, different ex post: their histories differ.
- Each household's problem is a **Markov decision process**: state $(a, \varepsilon)$, action $a'$, reward $u(c)$, discount $\beta$, transition $\pi$.
- **The new object is the cross-section: a stationary distribution $\lambda(a, \varepsilon)$ of households.**

© Zhigang Feng Redistribution prohibited without permission

Source: Aiyagari, "Uninsured idiosyncratic risk and aggregate saving", *Quarterly Journal of Economics* (1994).

# Equilibrium Is Where the Curves Cross

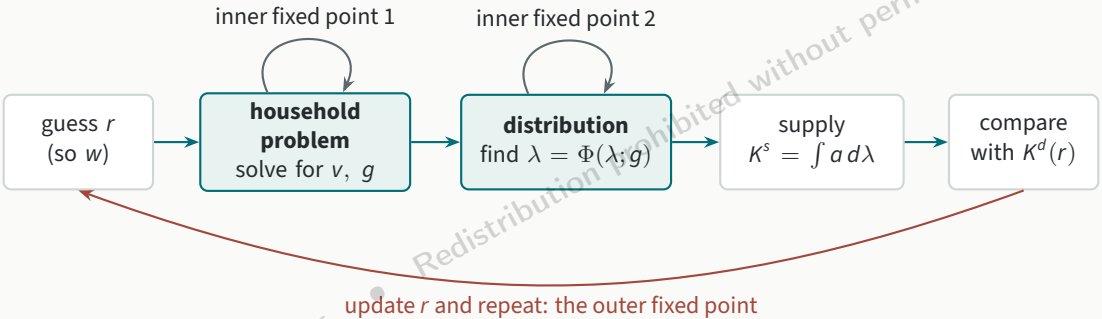



- $\rho = 1/\beta - 1$ is the rate of time preference. With full insurance, $r = \rho$.
- Savers facing risk would accumulate without bound as $r \to \rho$, so supply is finite only below it.
- Hence $r^* < \rho$, and capital sits above its complete-markets level.
- **That extra capital is not a free lunch. It is the cost of self-insurance.**

© Zhigang Feng Redistribution prohibited without permission

Source: Aiyagari (1994). A schematic: demand from the firm's first-order condition, supply asymptotic to $\rho = 1/\beta - 1$.

# The Nested Fixed Point




© Zhuoran Feng Redistribution prohibited without permission

- Every point on the supply curve contains two solved problems: a Bellman equation and an invariant distribution. Add a second price, or a tax, and the outer search becomes a system.
- **A small error at the outer level can hide a failure inside either inner problem.**

# Krusell–Smith: The Distribution Starts to Move

**Stationary world (Aiyagari).** The distribution $\lambda^*$ is an unknown we solve for *once*. Prices are numbers. Households forecast nothing.

add a TFP shock $z$

**With aggregate shocks.** The distribution $\Gamma_t$ is a *state that moves every period*, $\Gamma' = H(\Gamma, z, z')$. Prices move with it. Households must forecast a distribution.

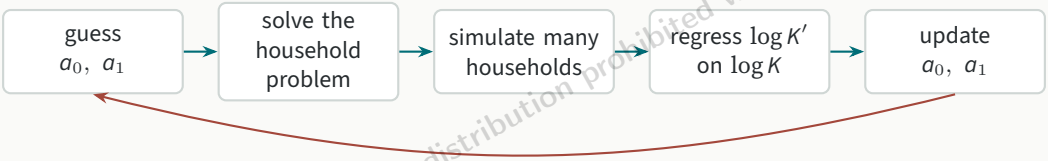
- $\Gamma$: the cross-sectional distribution of households over assets and productivity.
- Put wealth on 100 grid points and employment on 2. $\Gamma$ is then 200 numbers that sum to one: a point in a 199-dimensional simplex.
- In Aiyagari we solved for one such point. Here we need a *function* of it.
- **The state is no longer a vector. It is a distribution.**

Source: Krusell & Smith, "Income and wealth heterogeneity in the macroeconomy", *Journal of Political Economy* (1998).

© Zhigang Feng · Redistribution prohibited without permission

# The Krusell–Smith Bet

$$\log K' \;=\; a_0(z) + a_1(z)\,\log K$$

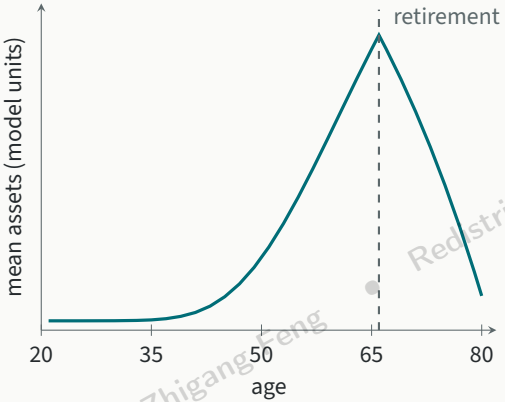



© Hugget Fear · Redistribution prohibited without permission

- The bet: households forecast with one moment, mean capital $K$, not all of $\Gamma$. The fit is $R^2 > 0.9999$: **approximate aggregation**.
- $R^2$ **can flatter.** It scores one-step forecasts; errors that build up over many periods hide behind it (Den Haan 2010).

Source: Krusell & Smith (1998); Den Haan, *Journal of Economic Dynamics and Control* (2010).

# OLG: Age Becomes a State

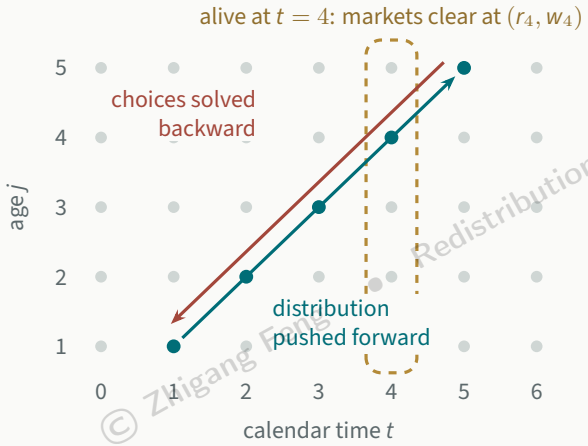



© Zhigang Feng
Redistribution prohibited without permission

- **Overlapping generations** (OLG): people live finite lives, and cohorts of different ages coexist.
- Sixty cohorts, ages 21 to 80, work to 65: they save for retirement, then run assets down.
- Finite lives help: solve backward from the last age, with no inner fixed point in values.
- **The state now carries age, so pensions, taxes and demographics can be studied.**

Source: After Auerbach & Kotlikoff (1987; 55 cohorts, to age 75). Profile: the instructor's 60-cohort model with a flat pension.

# Transitions: Households Backward, Distributions Forward



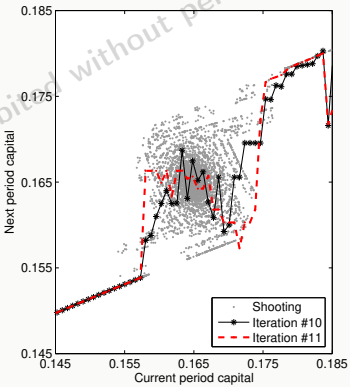

- A reform changes the whole price path $\{r_t, w_t\}$, not one number.
- Guess the path. Solve each cohort **backward** from its last age.
- Push the distribution **forward** from today's cross-section.
- Check market clearing in *every* period, update the path, repeat.
- **The unknown is a sequence of prices and distributions.**

© Zhigang Feng. Redistribution prohibited without permission

Source: Auerbach & Kotlikoff (1987); Conesa & Krueger, "Social security reform with heterogeneous agents", *RED* (1999).

# The Government Is a Player Too

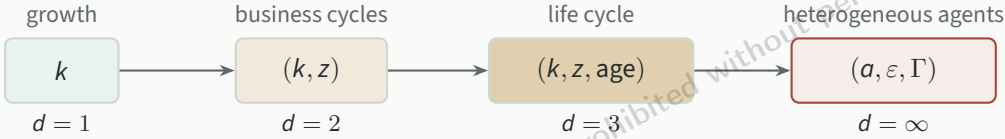
- **Ramsey policy**: choose the whole tax path at date 0. Households' Euler equations become constraints.
- **Commitment adds a state**: a past promise, with a feasible range unknown in advance.
- **Time inconsistency** (Kydland–Prescott 1977): tomorrow's government will not keep today's plan.
- **Theory warns us.** A continuous equilibrium can fail to exist. A method that assumes a smooth policy then returns a confident, wrong answer.




© Zhigang Peng · Distribution prohibited without permission

Source: Santos, *Journal of Economic Theory* (2002). Right: his example computed numerically; iterations do not settle.

# What Are We Solving For? The Objects Escalate




| Model | Object to compute | Computational problem |
|---|---|---|
| Growth | value $V(k)$ and policy $g(k)$ | one fixed point |
| Aiyagari | policies, a price $r$, a distribution $\lambda$ | a nested fixed point |
| Krusell–Smith | policies and a law $\Gamma' = H(\Gamma, z, z')$ | a fixed point in forecasting rules |
| OLG, Ramsey | policies by age, price paths, tax rules | backward and forward; a fixed point in rules |

© Zhigang Feng — Redistribution prohibited without permission

# Why Grids Break

**1 state: $10^2$ points**



**2 states: $10^4$ points**



**5 states: $10^{10}$ points**



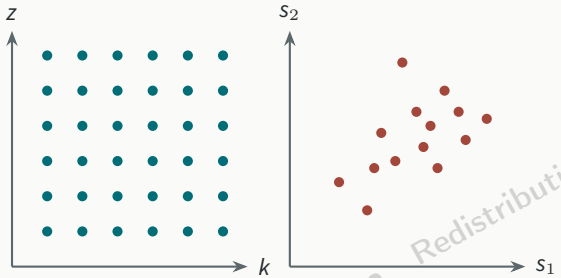
| states $d$ | grid points | one value table |
|---|---|---|
| 2 | $10^4$ | 0.08 MB |
| 4 | $10^8$ | 0.8 GB |
| 6 | $10^{12}$ | 8 TB |

- **Curse one, the grid:** 100 points per state gives $100^d$ points.
- **Curse two, the expectation:** with 7 nodes per shock, 3 shocks cost $7^3 = 343$ terms at every grid point, every iteration.
- **A distribution as a state makes $d$ effectively infinite.**

Source: Bellman (1957) coined "the curse of dimensionality". Dots are a schematic sample, not the literal count.

© Zhigang Feng. Redistribution prohibited without permission

# Where AI Enters (1): From Grids to Samples




**classical**
a value at every grid point

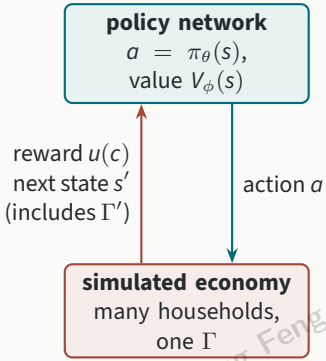
**learned**
a network fit on visited states

- A neural network stands in for the policy or value function.
- All equilibrium conditions enter one loss, minimised by gradient descent.
- Training states come from simulating the model: **you pay only for the states the economy visits.**
- So $\Gamma$ need not be collapsed to a few moments.

© Zhigang Feng
Redistribution prohibited without permission

Source: Maliar, Maliar & Winant, *JME* (2021); Azinovic, Gaegauf & Scheidegger, *IER* (2022); Han, Yang & E, *QE* (2026); Fernández-Villaverde, Nuño & Perla, NBER WP 33117 (2024).

# Where AI Enters (1), Continued: Reinforcement Learning




- **Critic**, target $\bar{\phi}$ held fixed:

$$\min_{\phi} \mathbb{E}\left[\left(u(c) + \beta V_{\bar{\phi}}(s') - V_\phi(s)\right)^2\right]$$

- **Actor**, along simulated paths:

$$\max_{\theta} \mathbb{E}\left[u(c_\theta) + \beta V_\phi(s'_\theta)\right]$$

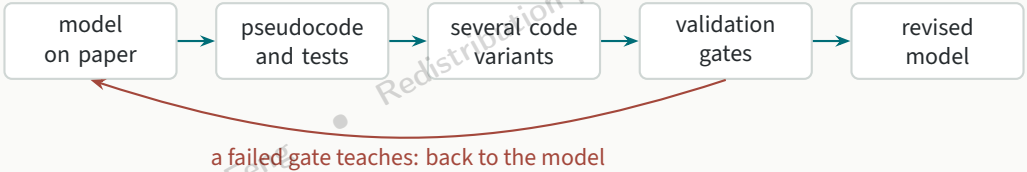
- No transition law is written down: expectations are simulated averages.
- **Equilibrium: every household uses the trained rule. Sessions 3 and 4 build it.**

Source: Sutton & Barto, *Reinforcement Learning* (2018), ch. 13; Miao & Feng, *AI for Economists*, ch. 11.

© Zhigang Feng
Redistribution prohibited without permission

# Where AI Enters (2) and (3): Data, and a Collaborator

- **(2) Unstructured data become measurement.** Central-bank text gives a policy-stance index, night lights proxy for output, newspapers yield an uncertainty index.
- **(3) AI as a collaborator** helps you understand and write models, design algorithms, and implement code. Session 6 develops this loop.

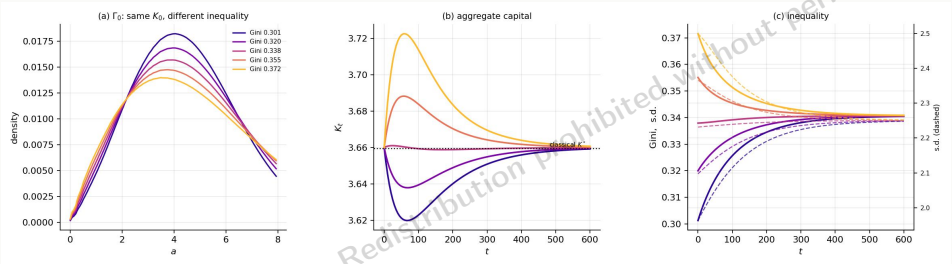



- **AI writes variants quickly. You choose the state, the equilibrium concept, the benchmark, and what a failure means.**

Source: Hansen, McMahon & Prat (2018); Henderson, Storeygard & Weil (2012); Baker, Bloom & Davis (2016).

© Shigang Feng. Redistribution prohibited without permission

# Research in Practice: Every Start Ends in the Same Place




© Zhiyang Feng. Redistribution prohibited without permission

- Five Aiyagari economies: the *same* $K_0 = 3.660$, Gini 0.301 to 0.372. All reach the classical $K^* = 3.6595$.
- A network trained on $\Gamma_{t+1} = \Phi(\Gamma_t)$ makes convergence a result, not an assumption.
- **Same $K_0$, yet the distributions move for hundreds of periods. $K$ alone misses it.**

Source: Feng, Han, Sargent & Zhu (2026), work in progress. Terminal $K$ agrees to $3.5 \times 10^{-8}$.

# What Stays With the Researcher

- A converged training loss certifies nothing. A solution is certified by a ladder of checks:
  - **Feasibility**: every choice respects the budget and borrowing constraints.
  - **Residuals on fresh draws**: unit-free Euler and Bellman errors, on states not used in training.
  - **A classical benchmark**: value function iteration on the low-dimensional case, where it still works.
  - **Theory**: policies are monotone, values concave, consumption smooth.
- The top rung is not computational: **your name on the result**.
- Today's lab climbs the first three rungs on the growth model, in under a minute.
- **Verification, judgment and responsibility stay with the researcher.**

Source: Feng, "AI for Economics and the Economics of AI", working paper (2026), §3.3; Judd, Maliar & Maliar (2017).

© Zhigang Feng — Redistribution prohibited without permission

# Where Each Model Returns

| Session | Object | What changes |
|---|---|---|
| 1 | Orientation | AI as tool, as object, and as an agent with incentives |
| 2 | Deep learning | Functions become trainable; equations become losses |
| 3 | Reinforcement learning | Sequential decisions are learned from interaction |
| 4 | Heterogeneous agents | Distributions and equilibrium become state variables |
| 5 | Text and LLMs | Language becomes data, measurement, and generation |
| 6 | RAG and agents | Retrieval, tools, traces, and governance in one workflow |

© Zhiyang Feng Redistribution prohibited without permission

- The growth model returns in Sessions 2 and 3; Aiyagari, Krusell–Smith, OLG and optimal taxation in Session 4.
- **Next: a first lab that solves the growth model with a network, and checks it. Deck 1.5 sets up the workbench it runs on.**

# Lab: Two Machines, One Environment




© Zhirang Feng
Redistribution prohibited without permission

- Machine A minimises the squared Euler residual of Approach 2.
- Machine B learned the *look* of FOMC statements, not facts: no residual to compute.
- **Same technology, two epistemic positions: one output you can verify, one you cannot.**

Session 1.5

# The Workbench: Python, VS Code, Git and GitHub

## From a clean laptop to Python code that runs again, on any machine

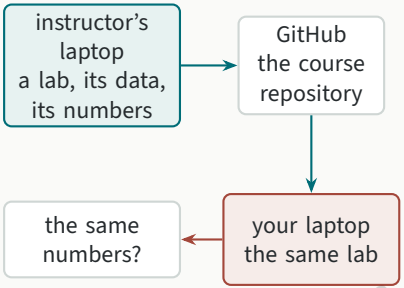



Redistribution prohibited without permission

Zhigang Feng

© Zhigang Feng

# Why a Workbench




the workbench decides whether the last arrow holds

- Every later session hands you a lab notebook that runs offline on a laptop CPU: no API key, no GPU.
- Whether it runs on *your* laptop depends on what you set up once: the course files, a Python, its packages, an editor, and a record of what you changed.
- This deck builds that, teaches the Python the labs are written in, and ends with the test: the Session 1 lab.
- **A result counts only if it runs again: tomorrow, on another machine, by someone else.**

© Zhigang Feng
Redistribution prohibited without permission

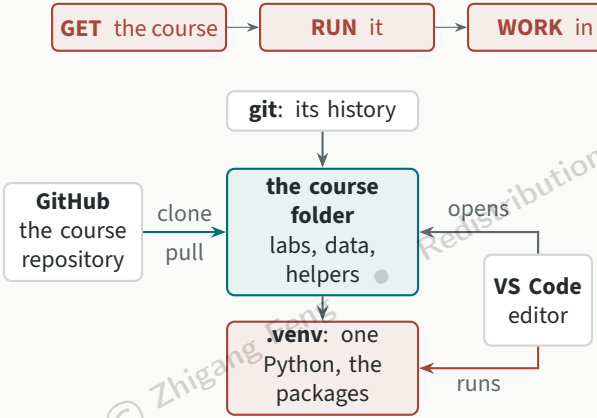
# Four Messages You Will Meet

| what you see | what it means | fixed in |
|---|---|---|
| `command not found: git` | the shell cannot find the program | GET |
| `ModuleNotFoundError: No module named 'torch'` | the package is not in the Python running your code | RUN, WORK |
| `NameError: name 'beta' is not defined` | a notebook depended on a cell it never reran | WORK |
| "It worked yesterday." | something changed, and nobody can say what | KEEP |

Redistribution prohibited without permission
© Zhiyuan Feng

- Four parts each remove one: GET the course, RUN it, WORK in it, KEEP it. A fifth, WRITE, teaches the Python itself.
- **None of the four is a Python problem; all four are workbench problems, and each costs more hours than the economics it blocks.**

# The Workbench in One Picture



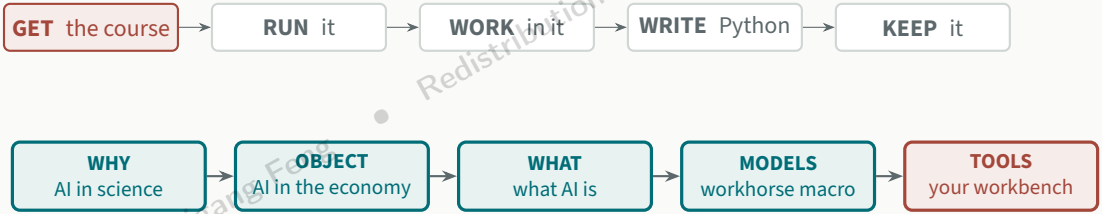

© Zhigang Feng
Redistribution prohibited without permission

- **One folder**, cloned from GitHub; **one environment** inside it; **one editor** pointed at both; **one history** watching the folder.
- The editor runs nothing itself: it hands your code to the Python it is pointed at.
- **The parts build these pieces in order; WRITE fills the folder with Python.**

Session 1.5

# Part I · GET the Course

Three installs, a terminal, and one clone




© Zhigang Feng Redistribution prohibited without permission

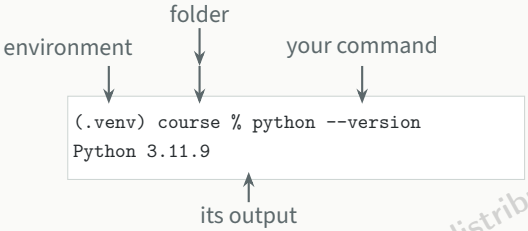
# Three Installs, Two Operating Systems

| | Windows | macOS |
|---|---|---|
| **Python 3.11** the interpreter | the 3.11.9 installer from python.org; tick *Add python.exe to PATH*, off by default | the 3.11.9 installer from python.org; the command is `python3` |
| **Git** the history tool | Git for Windows, from git-scm.com | the command-line tools: `xcode-select --install` |
| **VS Code** the editor | code.visualstudio.com | code.visualstudio.com |

© Zhiyang Feng Redistribution prohibited without permission

- Three separate programs: installing VS Code installs neither Python nor git. If python.org is slow, the Huawei Cloud mirror carries the same installers.
- **Three installs, done once, before Session 2; check each with `--version`.**

# The Terminal: Where Commands Go

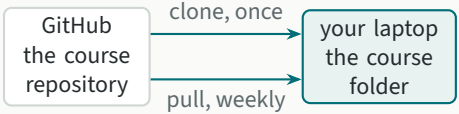



- A **shell** runs the commands you type: Terminal on macOS, PowerShell on Windows, or the terminal inside VS Code.
- `cd` enters a folder, `ls` (`dir`) lists it; `--version` asks a program which one it is.
- **PATH** is the list of folders searched for a program: `command not found` ("not recognized" on Windows) means it is on none of them.
- **Removes `command not found`: install, then open a new terminal.**

© Zhigang Feng
Redistribution prohibited without permission

# Clone the Course Once, Pull Every Week

```
git clone https://github.com/vonzhg/AI-ECON-2026.git     # once
cd AI-ECON-2026
git pull                                                  # before each session
```




the lab reads helpers and data from the folders beside it

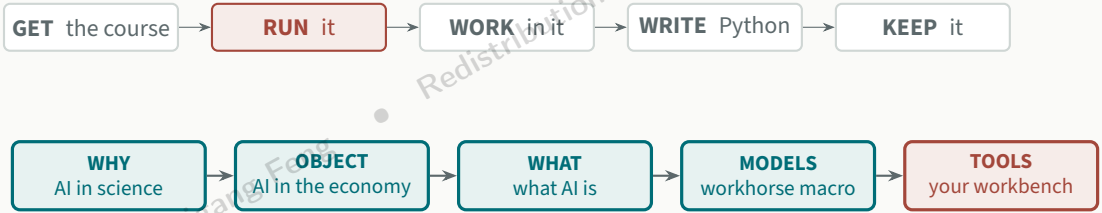
- Cloning a public repository needs no account; `git pull` brings each new lab.
- A notebook downloaded alone fails: it needs the helpers and data beside it.
- On a slow line, `git clone --depth 1` fetches only the latest version.
- Clone into `~/code`, not a OneDrive or iCloud folder (Part V).
- **The folder is the unit: clone it whole, update it with pull.**

© Zhigang Feng Redistribution prohibited without permission

Session 1.5

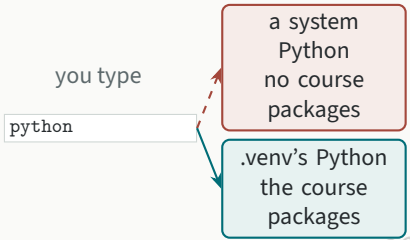
# Part II · RUN It

One Python, one environment, the course's packages




Redistribution prohibited without permission
© Zhigang Feng

# Which Python Is Running?




PATH is searched in order; activating `.venv` puts its Python first

- A laptop can carry several Pythons. A command runs the first one found on PATH.
- `python --version` says which version; `which python` (`where python` on Windows) says which folder it lives in. Inside the course it should be the folder's `.venv`.
- The prompt tells you too: `(.venv)` at its start means the environment is active.
- **Run both commands before asking anyone for help: every answer depends on which Python.**

© Zhigang Feng Redistribution prohibited without permission

# Environments: One Project, One Python, One Set of Versions

```
python3 -m venv .venv            # macOS: once, inside the course folder
source .venv/bin/activate        #        then in every new terminal
py -3.11 -m venv .venv           # Windows: once
.venv\Scripts\Activate.ps1       #          then in every new PowerShell
```

course folder
`.venv`: torch, numpy 2

another project
its own `.venv`: numpy 1

two environments never share a version, so they never fight

PowerShell refuses the script? Once:

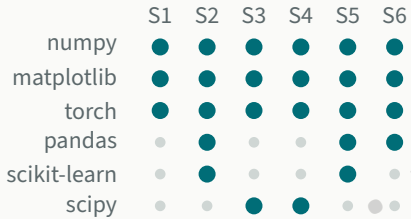
```
Set-ExecutionPolicy RemoteSigned
-Scope CurrentUser
```

- An **environment** is a folder holding one Python and one set of packages for one project.
- Install *into* the active one with `python -m pip`. Broken? Delete `.venv`, create it again.
- **Most `ModuleNotFoundError`s are a package installed into a different Python.**

Redistribution prohibited without permission
(C) Zhiheng Huang

# Install the Course's Packages, Once

```
python -m pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple   # once
python -m pip install numpy pandas matplotlib scipy scikit-learn torch jupyter ipykernel
```




which Session lab imports what; jupyter and ipykernel run the notebooks

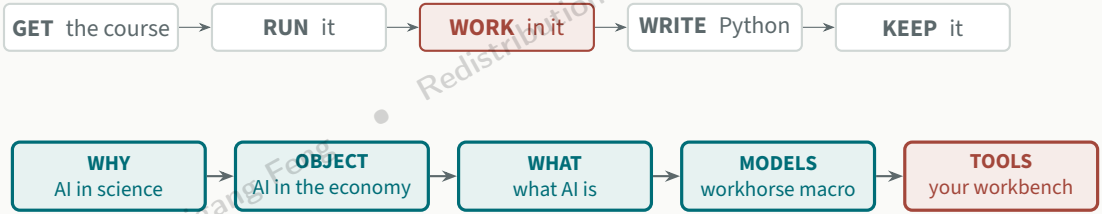
- The first line points pip at the Tsinghua mirror; the second installs all eight packages into the active `.venv`.
- The list names packages, not versions: the lab's first cell prints the versions you got. `torch` (PyTorch) is installed now and taught in Deck 2.4.
- **Removes `ModuleNotFoundError` at its source: everything the six labs import is now installed, in the Python they will use.**

© Zhigang Feng Redistribution prohibited without permission

Session 1.5

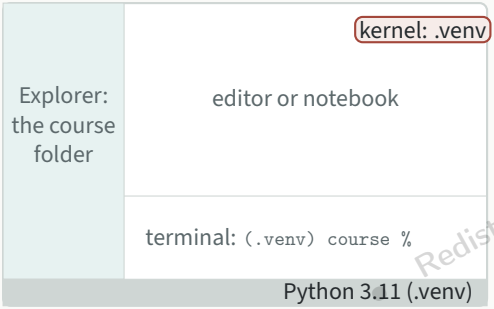
# Part III · WORK in It

VS Code, the kernel, and notebooks




© Zhigang Feng Redistribution prohibited without permission

# VS Code Runs Nothing Itself

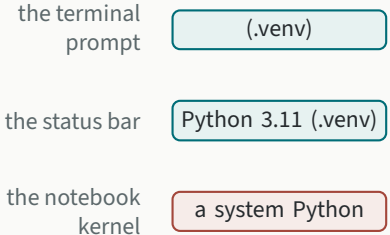



status bar: the interpreter the editor uses

- VS Code shows files and hands code to the Python you select. Install two extensions by Microsoft: *Python* and *Jupyter*.
- **File** → **Open Folder** on the course folder, not a single file; VS Code then finds its `.venv`.
- An AI assistant in the editor writes code; it does not run or check it (Deck 1.3; Session 6).
- **The editor is a window onto a folder and a Python. Get both right.**

© Zhigang Feng Redistribution prohibited without permission

# Three Places Choose a Python

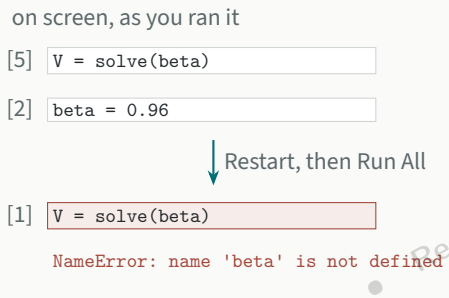



one of three on the wrong Python: the notebook cannot import torch

- All three must say `.venv`: the terminal you install in, the editor, and the notebook you run.
- **Python: Select Interpreter** sets the editor's; **Select Kernel** (a notebook's top right) sets the notebook's.
- A terminal opened inside VS Code activates the selected `.venv`; one opened elsewhere does not until you activate it.
- **Removes the other `ModuleNotFoundError`: installed, but not where the notebook looks.**

© Zhigang Feng Redistribution prohibited without permission

# A Notebook Is Not a Program



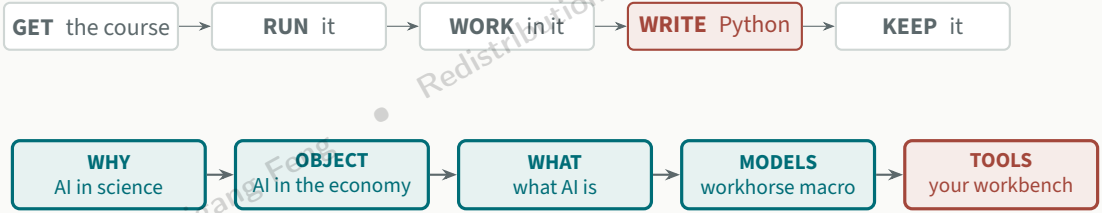

- A notebook keeps every variable until its kernel restarts, even ones whose cell you edited or deleted; the numbers in brackets record the order you ran things.
- So a notebook can look right and depend on a state nobody can rebuild.
- Before you trust or send a number: **Restart**, then **Run All**. A `NameError` then has found the hidden dependency.
- **Long or repeated runs go in a script that reads its settings at the top and writes results to disk.**

© Zhigang Feng Redistribution prohibited without permission

Session 1.5

# Part IV · WRITE Python

The language the labs are written in, on the lab's growth model




© Zhigang Feng
Redistribution prohibited without permission

# Values, Names and Types

```
>>> alpha = 0.33          # capital share
>>> beta = 0.96           # discount factor
>>> A = 1.0               # productivity
>>> k_ss = (alpha*beta*A) ** (1/(1-alpha))
>>> k_ss
0.17984701877776363
>>> type(alpha), type(200)
(<class 'float'>, <class 'int'>)
>>> print(f"steady state: {k_ss:.4f}")
steady state: 0.1798
```




- The lab's growth model: $k' = \alpha\beta A k^{\alpha}$, steady state $k^* = (\alpha\beta A)^{1/(1-\alpha)}$.
- `=` names a value; it is not an equation. `**` is a power.
- Types: `int`, `float`, text (`str`), `True`/`False` (`bool`); Python works them out.
- **A name points to a value; an f-string prints it.**

© Zhigang Feng Redistribution prohibited without permission

# Numbers Are Not Exact

```
>>> 0.1 + 0.2
0.30000000000000004
>>> 0.1 + 0.2 == 0.3
False
>>> import math
>>> math.isclose(0.1 + 0.2, 0.3)
True
>>> 1 / 2, 1 // 2, 7 // 2
(0.5, 0, 3)
```



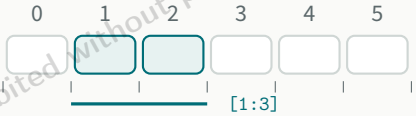

floats are a grid; 0.3 falls between two of its points

- A `float` carries about 16 significant digits, and most decimals are not exactly on its grid.
- Never test a computed float with `==`; compare with a tolerance, as `math.isclose` does.
- `//` divides and rounds down to an integer: how you turn a value into a grid position.
- **Sessions 2–4 judge a solution by residuals near $10^{-6}$: know that $10^{-16}$ is the floor.**

© Zhaonan Feng
Redistribution prohibited without permission

# Lists: A Path of Capital

```
>>> path = [0.5 * k_ss]          # start at k*/2
>>> for t in range(5):
...     path.append(alpha*beta*A*path[-1]**alpha)
...
>>> len(path)
6
>>> [round(k, 4) for k in path]
[0.0899, 0.1431, 0.1668, 0.1754, 0.1784, 0.1794]
>>> [round(k, 4) for k in path[1:3]]
[0.1431, 0.1668]
```




- A list keeps values in order; `append` adds at the end; `path[-1]` is the last.
- Positions start at 0; `[1:3]` stops *before* 3.
- `[f(x) for x in xs]` builds a list in one line. **An off-by-one index raises no error.**

© Zhigang Feng
No distribution prohibited without permission

# Dictionaries and Tuples: Parameters With Names

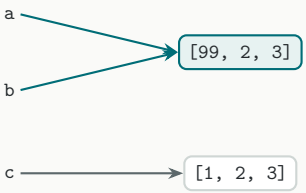
```
>>> params = dict(alpha=0.33, beta=0.96, A=1.0)
>>> params["beta"]
0.96
>>> list(params.keys())
['alpha', 'beta', 'A']
>>> s = params["alpha"] * params["beta"]
>>> y = params["A"] * k_ss ** params["alpha"]
>>> k_next, c = s * y, (1 - s) * y
>>> round(k_next, 4), round(c, 4)
(0.1798, 0.3879)
```




- A **dict** maps names to values: a calibration as one object, read by name.
- A **tuple** holds several values; `k_next, c = …` unpacks it.
- At $k^*$ the policy returns $k^*$ (0.1798): a check.
- **Name parameters in a dict, not in your memory.**

© Zhigang Feng
Redistribution prohibited without permission

# A Name Is Not a Copy




`b = a`: a second name; `c = a.copy()`: a second object

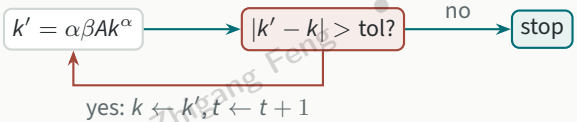
```
>>> a = [1, 2, 3]; b = a; b[0] = 99
>>> a
[99, 2, 3]
```

- Assignment never copies: `b = a` puts a second name on the same object. NumPy slices are *views* of the same memory too.
- **Value function iteration** repeats $V \leftarrow T(V)$ until the change $\max |V - V_{\text{old}}|$ is tiny.
- Write `V_old = V` and update `V` in place: both names see the update, the change is always 0, and the loop reports convergence after one step.
- **Write `V_old = V.copy()`: the bug is silent, the fix is one word.**

© Zhigang Feng
Redistribution prohibited without permission

# Control Flow: Iterate to the Steady State

```
>>> k, t, tol = 0.5 * k_ss, 0, 1e-8
>>> k_next = alpha*beta*A * k**alpha
>>> while abs(k_next - k) > tol:
...     k, t = k_next, t + 1
...     k_next = alpha*beta*A * k**alpha
...
>>> print(t, round(k_next, 6), round(k_ss, 6))
15 0.179847 0.179847
```

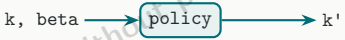



- `for t in range(5)` repeats a known number of times; `while` repeats until its test fails.
- The indented lines are the block that repeats; indentation is the syntax.
- From $k^*/2$ the loop stops after 15 steps, at $k^*$ to six decimals.
- **Every solver in this course stops this way: on a tolerance.**

© Zhiyang Feng Redistribution prohibited without permission

# Functions: Name the Rule Once

```
>>> def policy(k, alpha=0.33, beta=0.96, A=1.0):
...     "Next capital: alpha*beta*A*k**alpha."
...     return alpha * beta * A * k ** alpha
...
>>> round(policy(0.1), 4)
0.1482
>>> round(policy(0.1, beta=0.99), 4)
0.1528
>>> def step(k):
...     k_next = policy(k)
...     return k_next, k ** alpha - k_next
...
>>> [round(x, 4) for x in step(k_ss)]
[0.1798, 0.3879]
```




- `def` names a rule; `return` sends back the result; the string says what it does.
- Defaults (`beta=0.96`) change by name: `policy(0.1, beta=0.99)`.
- A function can return a tuple: $k'$ and consumption.
- **One rule, one definition, many calls.**

© Zhigang Feng
Redistribution prohibited without permission

# Modules and `import`

```
# growth.py, a file beside your notebook
ALPHA, BETA, A = 0.33, 0.96, 1.0

def policy(k):
    """Next period's capital."""
    return ALPHA * BETA * A * k ** ALPHA

if __name__ == "__main__":   # only as a script
    print(round(policy(0.1), 4))
```

```
>>> from growth import policy
>>> round(policy(0.1), 4)
0.1482
```




- A **module** is a file of Python; `import` runs it once and lends you its names. NumPy is a module: `import numpy as np`.
- The last two lines run only when the file is run as a script (`python growth.py`), not on import.
- **The labs import helpers from their folder: that is why you clone it whole.**

© Zhigang Feng
Redistribution prohibited without permission

# A Model as a Class

```
class GrowthModel:
    def __init__(self, alpha=0.33, beta=0.96, A=1.0):
        self.alpha, self.beta, self.A = alpha, beta, A
        self.k_ss = (alpha*beta*A) ** (1/(1-alpha))

    def policy(self, k):
        return self.alpha*self.beta*self.A*k**self.alpha

    def simulate(self, k0, T):
        path = [k0]
        for _ in range(T):
            path.append(self.policy(path[-1]))
        return path
```

```
>>> m = GrowthModel()
>>> round(m.k_ss, 4)
0.1798
>>> path = m.simulate(0.05, 30)
>>> round(path[-1], 4)
0.1798
```

- A **class** bundles parameters (`self.alpha`) with methods.
- `__init__` runs when you make one: `GrowthModel(beta=0.99)` is another economy.
- **The lab's networks are classes like this.**

© Zhigang Feng · Redistribution prohibited without permission

# Reading an Error

```
>>> policy("0.2")
Traceback (most recent call last):
  File "<console>", line 1, in <module>
  File "<console>", line 3, in policy
TypeError: unsupported operand type(s) for ** or pow(): 'str' and 'float'
>>> params["Beta"]
Traceback (most recent call last):
    ...
KeyError: 'Beta'
```

above: where, in your code

**last line: what went wrong**

- `NameError`: never defined. `TypeError`: wrong kind of value. `IndexError`: past the end. `KeyError`: not in the dict.
- **Read the last line first; paste the whole message when you ask for help.**

© Zhigang Feng
Redistribution prohibited without permission

# NumPy: One Array, No Loop

```
>>> import numpy as np
>>> k = np.linspace(0.2*k_ss, 2*k_ss, 200)
>>> k.shape, k.dtype
((200,), dtype('float64'))
>>> k_next = alpha*beta*A * k**alpha
>>> k_next[:3].round(4)
array([0.1057, 0.1073, 0.1088])
>>> loop = [policy(x) for x in k]
>>> np.allclose(k_next, loop)
True
```

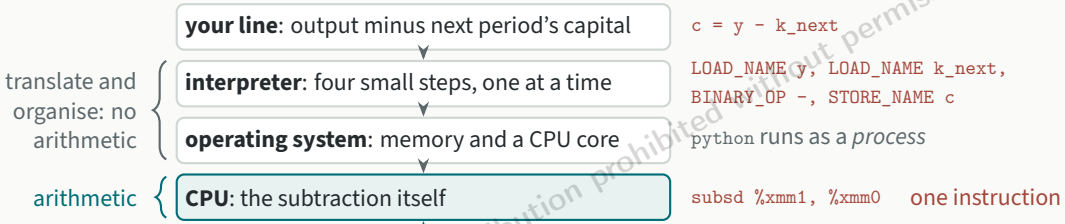



one block of 200 float64 numbers: a grid on $k$

- An **array** holds numbers of one type in one block of memory. `linspace` makes a grid of 200 points from $0.2k^*$ to $2k^*$.
- Arithmetic on an array applies to every element: one line computes the policy at all 200 points.
- The list loop gives the same numbers; NumPy runs its loop in compiled code, far faster.
- **On a grid, write the formula once and let the array carry it.**

© Zhigang Feng Redistribution prohibited without permission

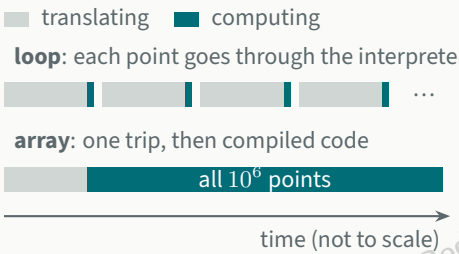
# One Step Deeper: What Happens to One Line

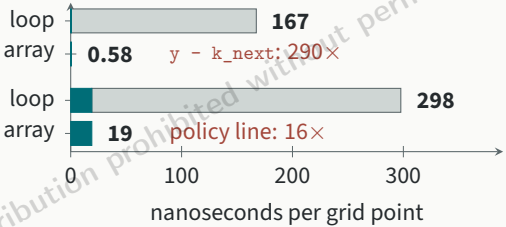



© Zijian Cheng · Distribution prohibited without permission

- The **interpreter** (`python`) is a live translator: each line becomes small steps (**bytecode**), run one by one, every time the code runs.
- **Machine code** fits one operating system and one CPU family (Intel/AMD or Apple silicon): build a `.venv` on each machine; never copy one.
- **Only the CPU computes; every layer above it translates or organises.**

# Where the Time Goes: Translating Versus Computing




© Zhiyuan Chen
Redistribution prohibited without permission

- Timed on $10^6$ grid points, best of 7: `c = y - k_next` takes 167 ns a point in a loop and 0.58 ns as an array; the NumPy frame's policy line, 298 against 19 ns.
- Teal is the array's time, the arithmetic itself; grey is what the loop adds. A power is real work, so there the gap is 16 times, not 290.
- Measured on one Linux machine: run `%timeit` to see yours.
- **A loop pays the translation once per point; an array pays it once per call.**

# Shapes and Broadcasting

```
>>> a = np.ones(3)           # shape (3,)
>>> b = np.ones((3, 1))      # shape (3, 1)
>>> (a + b).shape
(3, 3)
>>> d = k[:, None] - k[None, :]
>>> d.shape
(200, 200)
>>> assert k_next.shape == k.shape
```



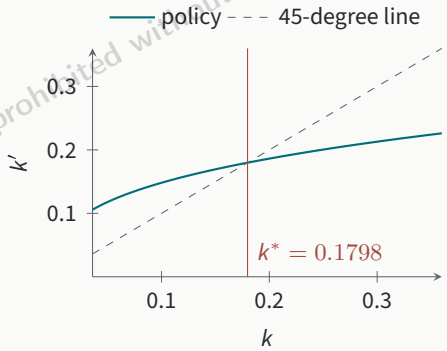

- **Broadcasting**: an axis of length 1 is stretched to match. `k[:, None] - k[None, :]` is every $k - k'$ on the grid at once, the matrix Session 2's solvers search.
- The trap: shapes (3,) and (3,1) give a (3,3) table, not three sums, and no error.
- PyTorch's tensors follow the same rules (Deck 2.4).
- **Assert the shape you expect.**

© Zhigang Feng Redistribution prohibited without permission

# Plot It: The Policy Against the 45-Degree Line

```
import matplotlib.pyplot as plt
plt.plot(k, k_next, label="policy")
plt.plot(k, k, "--", label="45-degree line")
plt.axvline(k_ss, color="grey")
plt.xlabel("k"); plt.ylabel("k'")
plt.legend()
plt.savefig("policy.pdf")    # before show()
plt.show()
```

- One line per curve; `savefig` writes the file a paper uses.
- **A figure your code can redraw is a result; a screenshot is not.**

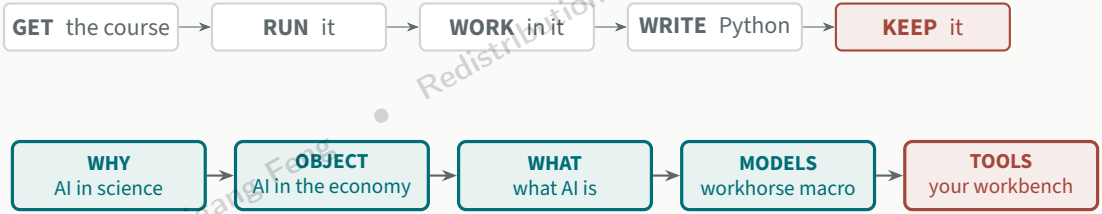



© Zejiang Feng · Redistribution prohibited without permission

Session 1.5

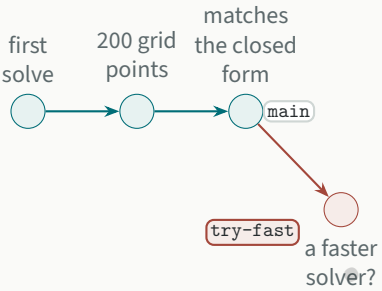
# Part V · KEEP It

A history of your work, and a copy on GitHub




© Zhigang Feng
Redistribution prohibited without permission

# Git: A History of Your Project




- **Git** keeps the history of a project folder. A **commit** is a snapshot of every tracked file, with a message and a link to the commit before it.
- The **history** is that chain; any commit can be compared with today or brought back. A **branch** is a line of commits you may throw away.
- A **repository** is the folder plus its history; every clone carries all of it.
- Git is not GitHub: git runs offline; GitHub keeps a copy online (Chacon & Straub, *Pro Git*, 2014).
- **Removes "it worked yesterday": every committed state can be recovered.**

© Zhigang Feng Redistribution prohibited without permission

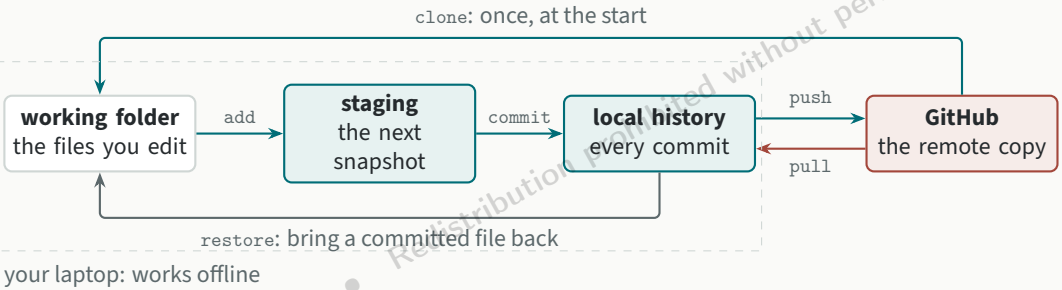
# What Git Can Do for You

| you want to | git does it with | where you meet it here |
|---|---|---|
| see what changed | `git status, git diff` | before every commit |
| record a result | `git add, git commit` | each time a result changes |
| find when it broke, go back | `git log, git diff <commit>, git restore` | “it worked yesterday” |
| try an idea safely | `git switch -c <branch>, git merge` | a new solver, kept apart |
| get the course, share your work | `git clone, git pull, git push` | weekly; your own repository |
| undo what an AI agent changed | `git diff, git restore` | Session 6, under an agent |

© Zhiqiang Feng Redistribution prohibited without permission

- `diff` shows changes line by line; `restore` brings back a file’s last committed version; `switch -c` starts a new branch; `merge` joins it back.
- **One idea under all six rows: every commit is kept. Nothing you committed is lost, only what you never committed.**

# Three Places on Your Machine, One on GitHub




© Ai-ming Feng Redistribution prohibited without permission

- `clone` copies the whole repository once. You edit in the working folder; `add` chooses the changes for the next snapshot; `commit` writes it into the history.
- `push` and `pull` exchange with GitHub: your history and GitHub's meet only then.
- **A commit is local until you push: until then it lives on one disk only.**

# One File, Three Edits: Before and After You Commit

Three edits to the committed `run.py`: `N` to 400, `tol` to `1e-8`, and a typo, `beta` to `9.6`.

**Save three times, commit later**

```
$ git status --short
 M run.py
$ git diff
...
-beta = 0.96
-N = 200
-tol = 1e-6
+beta = 9.6
+N = 400
+tol = 1e-8
```

Git does not see saves: one tangled change; `git restore` would undo all three.

**Commit after each save**

```
$ git log --oneline
018f80b update beta
2d58eb9 tol 1e-8
df73dcd grid N=400
7ab99ee first VFI: 200 points
$ git revert --no-edit HEAD
[main 5cd5716] Revert "update beta"
...
```

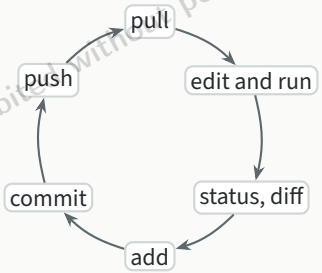
The typo sits alone in one commit; the revert restores `beta = 0.96` and keeps the two good edits.

- **A save changes the file; only a commit changes the history.**

© Zhirang Feng · Redistribution prohibited without permission

# The Daily Loop

```
git config --global user.name "Your Name"   # once
git config --global user.email "you@uni.edu"
git pull          # start from the latest copy
git status        # what changed?
git diff          # how, line by line?
git add run.py
git commit -m "VFI on 200 points: max Euler error 1.2e-4"
git push          # your history -> GitHub
git log --oneline # the story so far
```




© Liyang Feng · Redistribution prohibited without permission

- Tell git who you are once; then, in your own repository, loop: pull, look at what changed, choose it, commit it with a message, push. The course folder only pulls.
- **Commit when a result changes, and put the result in the message: it is a note to yourself in three months.**

# When git pull Refuses

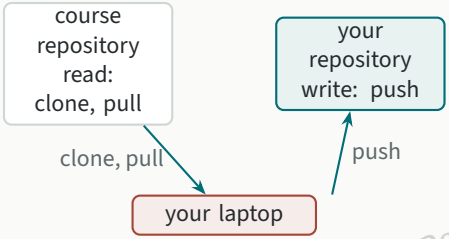
```
$ git pull
error: Your local changes to the following files would be overwritten by merge:
        lab.ipynb
Please commit your changes or stash them before you merge.
Aborting
```




- Running a notebook saves its outputs into the file, so git sees a change you never typed, and the next pull stops rather than overwrite it.
- To keep your run: save it under your own name first. To discard it: `git restore lab.ipynb`, then pull.
- **Run your own copy; let pull own the course's files.**

© Zhigang Feng Redistribution prohibited without permission

# GitHub: Your Own Repository




one laptop, two remotes: one you read, one you write

- Make a GitHub account; create a repository for your own work and clone it, the verb from Part I.
- Pushing needs a sign-in, and GitHub has not accepted account passwords for git since 2021: use an SSH key, or sign in once with GitHub's `gh auth login`.
- If a laptop install fails, GitHub Codespaces runs the same notebooks in a browser.
- **Read from the course, write to your own.**

© Zhigang Feng
Redistribution prohibited without permission

# What Never Goes Into a Repository

**commit**
code
the package list
a README: how to run it

**never**
data you do not own
model checkpoints
keys, passwords
`.venv`

```
# .gitignore
.venv/
*.pt
.env
```

- A key pushed once is public, and stays in the history after you delete the file.
- A `.gitignore` lists what git must never track; write it once, at the start.
- What always goes in is what someone else needs to rerun you: code, the package list, a README.
- **In Session 6 git is also the safety net under an AI agent: the habits start here.**

© Zhigang Feng
Redistribution prohibited without permission

# Keep the Repository Out of OneDrive and iCloud

| | a sync folder | git |
|---|---|---|
| what it saves | each file, alone | the whole project |
| when | on every change | when you commit |
| a message | none | one per commit |
| same file, two machines | keeps both, one renamed | `pull` merges, or shows the clash |

one system per folder: for code, git, with GitHub as the copy

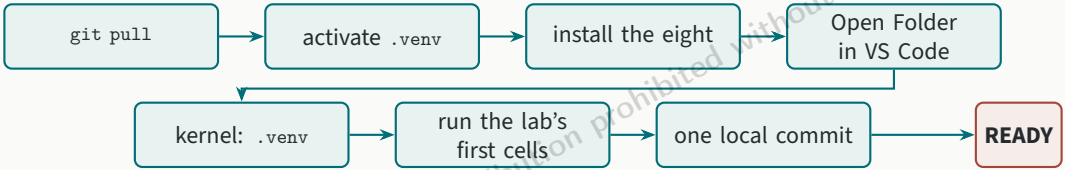
**What git says when `.git/` arrives half-synced:**

```
fatal: Unable to create '.../.git/index.lock': File exists.
fatal: bad object HEAD
error: bad signature 0x00000000
fatal: index file corrupt
```

- Windows often moves Documents and Desktop into OneDrive; macOS can put them in iCloud Drive.
- `.git/` is many small files that change together; a sync copies them one at a time (left).
- "Online-only" files are placeholders git cannot read.
- **Keep a repository in a plain folder such as `~/code`, outside any sync.**

© Zhigong Feng · Redistribution prohibited without permission

# Ready: The Check Before Session 2




© Zhigang Feng · Redistribution prohibited without permission

- The lab's first cells print Python, every package's version and git, then find the course folder; a missing piece is named, with what to install.
- A commit of any small file in your own repository shows that git works; it needs no account.
- **If every step prints, the rest of the course runs on your laptop.**

# Open the Lab, and the Bridge to Session 2

- Open the Session 1 lab notebook in the course folder, select the `.venv` kernel, and run the cells from the top. Its first cells are the check on the last slide.
- Two models, both offline on a laptop CPU with fixed seeds: a neural network that learns the growth model's saving policy and is checked against the closed form, and a small text generator trained on FOMC statements, with no benchmark inside its text.
- Fifteen minutes: the check, the two models, then three temperatures (0.5, 0.8, 1.2).
- Note privately what changed when you turned the dial: fluency, repetition, reliability.
- **Session 2 opens the first model: from one neuron to a trained network. PyTorch, NumPy's arrays plus derivatives computed for you, is Deck 2.4.**

© Zhiqiang Feng · Redistribution prohibited without permission